

Hutch++: Optimal Stochastic Trace Estimation

Raphael A. Meyer
New York University
ram900@nyu.edu

Cameron Musco
UMass Amherst
cmusco@cs.umass.edu

Christopher Musco
New York University
cmusco@nyu.edu

David P. Woodruff
Carnegie Mellon University
dwoodruf@cs.cmu.edu

June 10, 2021

Abstract

We study the problem of estimating the trace of a matrix $\mathbf{A}$ that can only be accessed through matrix-vector multiplication. We introduce a new randomized algorithm, Hutch++, which computes a $(1 \pm \varepsilon)$ approximation to $\text{tr}(\mathbf{A})$ for any positive semidefinite (PSD) $\mathbf{A}$ using just $O(1/\varepsilon)$ matrix-vector products. This improves on the ubiquitous *Hutchinson's estimator*, which requires $O(1/\varepsilon^2)$ matrix-vector products. Our approach is based on a simple technique for reducing the variance of Hutchinson's estimator using a low-rank approximation step, and is easy to implement and analyze. Moreover, we prove that, up to a logarithmic factor, the complexity of Hutch++ is optimal amongst all matrix-vector query algorithms, even when queries can be chosen adaptively. We show that it significantly outperforms Hutchinson's method in experiments. While our theory mainly requires $\mathbf{A}$ to be positive semidefinite, we provide generalized guarantees for general square matrices, and show empirical gains in such applications.

1 Introduction

A ubiquitous problem in numerical linear algebra is that of approximating the trace of a $d \times d$ matrix $\mathbf{A}$ that can only be accessed via matrix-vector multiplication queries. In other words, we are given access to an oracle that can evaluate $\mathbf{A}\mathbf{x}$ for any $\mathbf{x} \in \mathbb{R}^d$, and the goal is to return an approximation to $\text{tr}(\mathbf{A})$ using as few queries to this oracle as possible. An exact solution can be obtained with d queries because $\text{tr}(\mathbf{A}) = \sum_{i=1}^d \mathbf{A}_{ii} = \sum_{i=1}^d \mathbf{e}_i^T \mathbf{A} \mathbf{e}_i$, where $\mathbf{e}_i$ denotes the i^{th} standard basis vector. The goal is thus to develop algorithms that use far fewer than d matrix-vector multiplications.

Known as *implicit* or *matrix free* trace estimation, this problem arises in applications that require the trace of a matrix $\mathbf{A}$, where $\mathbf{A}$ is itself a transformation of some other matrix $\mathbf{B}$. For example, $\mathbf{A} = \mathbf{B}^q$, $\mathbf{A} = \mathbf{B}^{-1}$, or $\mathbf{A} = \exp(\mathbf{B})$. In all of these cases, explicitly computing $\mathbf{A}$ would require roughly $O(d^3)$ time, whereas multiplication with a vector $\mathbf{x}$ can be implemented more quickly using iterative methods. For example, $\mathbf{B}^q \mathbf{x}$ can be computed in just $O(d^2)$ time for constant q , and for well-conditioned matrices, $\mathbf{B}^{-1} \mathbf{x}$ and $\exp(\mathbf{B}) \mathbf{x}$ can also be computed in $O(d^2)$ time using the conjugate gradient or Lanczos methods [Hig08]. Implicit trace estimation is used to approximate matrix norms [HMAS17, MNS⁺18], spectral densities [LSY16, CKSV18, BKKS20], log-determinants [BDKZ15, HMS15], the Estrada index [US18, WSMB20], eigenvalue counts in intervals [DNPS16], triangle counts in graphs [Avr10], and much more [Che16]. In these applications, we typically have that $\mathbf{A}$ is symmetric, and often positive semidefinite (PSD).

1.1 Hutchinson’s Estimator

The most common method for implicit trace estimation is Hutchinson’s stochastic estimator [Hut90]. This elegant randomized algorithm works as follows: let $\mathbf{G} = [\mathbf{g}_1, \dots, \mathbf{g}_m] \in \mathbb{R}^{d \times m}$ be a matrix containing i.i.d. random variables with mean 0 and variance 1. A simple calculation shows that $\mathbb{E}[\mathbf{g}_i^T \mathbf{A} \mathbf{g}_i] = \text{tr}(\mathbf{A})$ for each $\mathbf{g}_i \in \mathbb{R}^d$, and $\mathbf{g}_i^T \mathbf{A} \mathbf{g}_i$ can be computed with just one matrix-vector multiplication. So to approximate $\text{tr}(\mathbf{A})$, Hutchinson’s estimator returns the following average:

$$\text{Hutchinson’s Estimator:} \quad H_m(\mathbf{A}) = \frac{1}{m} \sum_{i=1}^m \mathbf{g}_i^T \mathbf{A} \mathbf{g}_i = \frac{1}{m} \text{tr}(\mathbf{G}^T \mathbf{A} \mathbf{G}). \quad (1)$$

Hutchinson’s original work suggests using random ± 1 sign vectors for $\mathbf{g}_1, \dots, \mathbf{g}_m$, and an earlier paper by Girard suggests standard normal random variables [Gir87]. Both choices perform similarly, as both random variables are sub-Gaussian. For vectors with sub-Gaussian random entries, it can be proven that, when $\mathbf{A}$ is positive semidefinite, $(1 - \varepsilon) \text{tr}(\mathbf{A}) \leq H_m(\mathbf{A}) \leq (1 + \varepsilon) \text{tr}(\mathbf{A})$ with probability $\geq 1 - \delta$ if we use $m = O(\log(1/\delta)/\varepsilon^2)$ matrix-vector multiplication queries [AT11, RA15].¹ For constant δ (e.g., $\delta = 1/10$) the bound is $O(1/\varepsilon^2)$.

1.2 Our results

Since Hutchinson’s work, and the non-asymptotic analysis in [AT11], there has been no improvement on this $O(1/\varepsilon^2)$ matrix-vector multiplication bound for trace approximation. Our main contribution is a quadratic improvement: we provide a new algorithm, Hutch++, that obtains the same $(1 \pm \varepsilon)$ guarantee with $O(1/\varepsilon)$ matrix-vector multiplication queries. This algorithm is nearly as simple as the original Hutchinson’s method, and can be implemented in just a few lines of code.

Algorithm 1 Hutch++

input: Matrix-vector multiplication oracle for matrix, $\mathbf{A} \in \mathbb{R}^{d \times d}$. Number of queries, m .

output: Approximation to $\text{tr}(\mathbf{A})$.

- 1: Sample $\mathbf{S} \in \mathbb{R}^{d \times \frac{m}{3}}$ and $\mathbf{G} \in \mathbb{R}^{d \times \frac{m}{3}}$ with i.i.d. $\{+1, -1\}$ entries.
 - 2: Compute an orthonormal basis $\mathbf{Q} \in \mathbb{R}^{d \times \frac{m}{3}}$ for the span of $\mathbf{A} \mathbf{S}$ (e.g., via QR decomposition).
 - 3: **return** $\text{Hutch++}(\mathbf{A}) = \text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + \frac{3}{m} \text{tr}(\mathbf{G}^T (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{G})$.
-

Hutch++ requires m matrix-vector multiplications with $\mathbf{A}$: $m/3$ to compute $\mathbf{A} \cdot \mathbf{S}$, $m/3$ to compute $\mathbf{A} \cdot \mathbf{Q}$, and $m/3$ to compute $\mathbf{A} \cdot (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{G}$. It requires $O(dm^2)$ additional runtime to compute the basis $\mathbf{Q}$ and the product $(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{G} = \mathbf{G} - \mathbf{Q} \mathbf{Q}^T \mathbf{G}$. For concreteness, we state the method with random sign matrices, but the entries of $\mathbf{S}$ and $\mathbf{G}$ can be any sub-Gaussian random variables with mean 0 and variance 1, including e.g., standard Gaussians. Our main theorem on Hutch++ is:

Theorem 1. *If Hutch++ is implemented with $m = O(\sqrt{\log(1/\delta)}/\varepsilon + \log(1/\delta))$ matrix-vector multiplication queries, then for any PSD $\mathbf{A}$, with probability $\geq 1 - \delta$, the output $\text{Hutch++}(\mathbf{A})$ satisfies:*

$$(1 - \varepsilon) \text{tr}(\mathbf{A}) \leq \text{Hutch++}(\mathbf{A}) \leq (1 + \varepsilon) \text{tr}(\mathbf{A}).$$

Hutch++ can be viewed as a natural *variance reduced* version of Hutchinson’s estimator. The method starts by computing an orthonormal span $\mathbf{Q} \in \mathbb{R}^{d \times \frac{m}{3}}$ by running a single iteration of power

¹For non-PSD matrices, this generalizes to $\text{tr}(\mathbf{A}) - \varepsilon \|\mathbf{A}\|_F \leq H_m(\mathbf{A}) \leq \text{tr}(\mathbf{A}) + \varepsilon \|\mathbf{A}\|_F$, which implies the relative error bound since when $\mathbf{A}$ is PSD, $\|\mathbf{A}\|_F \leq \text{tr}(\mathbf{A})$.

method with a random start matrix $\mathbf{S}$. $\mathbf{Q}$ coarsely approximates the span of $\mathbf{A}$'s top eigenvectors. Then we separate $\mathbf{A}$ into its projection onto the subspace spanned by $\mathbf{Q}$, and onto that subspace's orthogonal complement, writing $\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{Q}\mathbf{Q}^T\mathbf{A}\mathbf{Q}\mathbf{Q}^T) + \text{tr}((\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\mathbf{A}(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T))$. By the cyclic property of the trace, the first term is equal to $\text{tr}(\mathbf{Q}^T\mathbf{A}\mathbf{Q})$, which is computed *exactly* by Hutch++ with $\frac{m}{3}$ matrix-vector multiplications. The second term is approximated using Hutchinson's estimator with the random vectors in $\mathbf{G}$.

Thus, the error in estimating $\text{tr}(\mathbf{A})$ is entirely due to approximating this second term. The key observation is that the variance when estimating this term is much lower than when estimating $\text{tr}(\mathbf{A})$ directly. Specifically, it is proportional to $\|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\mathbf{A}(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\|_F^2$, which, using standard tools from randomized linear algebra [CEM⁺15, Woo14], we can show is bounded by $\varepsilon \text{tr}(\mathbf{A})^2$ with good probability when $m = O(1/\varepsilon)$. This yields our improvement over Hutchinson's method applied directly to $\mathbf{A}$, which has variance bounded by $\text{tr}(\mathbf{A})^2$. The full proof of [Theorem 1](#) is in [Section 3](#).

[Algorithm 1](#) is *adaptive*: it multiplies $\mathbf{A}$ by a sequence of query vectors $\mathbf{r}_1, \dots, \mathbf{r}_m$, where later queries depend on earlier ones. In contrast, Hutchinson's method is *non-adaptive*: $\mathbf{r}_1, \dots, \mathbf{r}_m$ are chosen in advance, before computing any of the products $\mathbf{A}\mathbf{r}_1, \dots, \mathbf{A}\mathbf{r}_m$. In addition to [Algorithm 1](#), we give a non-adaptive variant of Hutch++ that obtains the same $O(1/\varepsilon)$ bound. We complement these results with a nearly matching lower bound, proven in [Section 4](#). Specifically, via a reduction from the Gap-Hamming problem from communication complexity, we show that any matrix-vector query algorithm whose queries have bounded bit complexity requires $m = \Omega\left(\frac{1}{\varepsilon \log(1/\varepsilon)}\right)$ queries to estimate the trace of a PSD matrix up to a $(1 \pm \varepsilon)$ multiplicative approximation. We also prove a tight $m = \Omega\left(\frac{1}{\varepsilon}\right)$ lower bound for non-adaptive algorithms in the real RAM model of computation.

Finally, we provide a generalization of our upper bound to non-PSD matrices. We also provide an analysis of the variance the Hutch++ estimator, which complements the high-probability bound of [Theorem 1](#). The variance analysis involves explicit constants, which may be useful to practitioners.

Empirical Results. In [Section 6](#) we present experimental results on synthetic and real-world matrices, including applications of trace estimation to approximating log determinants, the graph Estrada index, and the number of triangles in a graph. We demonstrate that Hutch++ improves substantially on Hutchinson's estimator, and on related estimators based on approximating the top eigenvalues of $\mathbf{A}$. While our relative error bounds only apply to PSD matrices, Hutch++ can be used unmodified on general square matrices, and we experimentally confirm that it still outperforms Hutchinson's in this case. We note that Hutch++ is simple to implement and essentially parameter free: the only choice needed is the number of matrix-vector multiplication queries m .

1.3 Prior Work

Upper bounds. A nearly tight non-asymptotic analysis of Hutchinson's estimator for positive semidefinite matrices was given by Avron and Toledo using an approach based on reducing to Johnson-Lindenstrauss random projection [AT11, DG03, Ach03]. A slightly tighter approach from [RA15] obtains a $(1 \pm \varepsilon)$ multiplicative error bound with $m = O(1/\varepsilon^2)$ matrix-vector multiplication queries. This bound is what we improve on with Hutch++. A more in depth discussion of different variations on Hutchinson's method and existing error bounds can be found in survey of [MT20].

A number of papers suggest variance reduction schemes for Hutchinson's estimator. Some take advantage of sparsity structure in $\mathbf{A}$ [TS11, SLO13] and others use a "decomposition" approach similar to Hutch++ [APJ⁺18]. Most related to our work are two papers which, like Hutch++, perform the decomposition by projecting onto some $\mathbf{Q}$ that approximately spans $\mathbf{A}$'s top eigenspace [GSO17, Lin17]. The justification is that this method should perform much better than Hutchinson's when $\mathbf{A}$ is close to low-rank, because $\text{tr}(\mathbf{Q}^T\mathbf{A}\mathbf{Q})$ will capture most of $\mathbf{A}$'s trace. Our con-

tribution is an analysis of this approach which 1) improves on Hutchinson’s *even when $\mathbf{A}$ is far from low-rank* and 2) shows that a very coarse approximation to the top eigenvectors suffices (computed using one iteration of the power method). Finally, we note two papers which directly use the approximation $\text{tr}(\mathbf{A}) \approx \text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q})$, where $\mathbf{Q}$ is computed with a randomized SVD method [SAI17, LZ21]. Of course, this approach works best for nearly-low rank matrices.

Lower bounds. Our lower bounds extend a recent line of work on lower bounds for linear algebra problems in the “matrix-vector query model” [SEAR18, SWYZ19, BHSW20]. [WWZ14] proves a lower bound of $\Omega(1/\varepsilon^2)$ queries for PSD trace approximation in an alternative model that allows for adaptive “quadratic form” queries: $\mathbf{r}_1^T \mathbf{A} \mathbf{r}_1, \dots, \mathbf{r}_m^T \mathbf{A} \mathbf{r}_m$. This model captures Hutchinson’s estimator, but not Hutch++, which is why we are able to obtain an upper bound of $O(1/\varepsilon)$ queries.

2 Preliminaries

Notation. For $\mathbf{a} \in \mathbb{R}^d$, $\|\mathbf{a}\|_2 = (\sum_{i=1}^d a_i^2)^{1/2}$ denotes the ℓ_2 norm and $\|\mathbf{a}\|_1 = \sum_{i=1}^d |a_i|$ denotes the ℓ_1 norm. For $\mathbf{A} \in \mathbb{R}^{n \times d}$, $\|\mathbf{A}\|_F = (\sum_{i=1}^n \sum_{j=1}^d \mathbf{A}_{ij}^2)^{1/2}$ denotes the Frobenius norm. For square $\mathbf{A} \in \mathbb{R}^{d \times d}$, $\text{tr}(\mathbf{A}) = \sum_{i=1}^d \mathbf{A}_{ii}$ denotes the trace. Our main results on trace approximation are proven for symmetric positive semidefinite (PSD) matrices, which are the focus of many applications. Any symmetric $\mathbf{A} \in \mathbb{R}^{d \times d}$ has eigendecomposition $\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^T$, where $\mathbf{V} \in \mathbb{R}^{d \times d}$ is orthogonal and $\mathbf{\Lambda}$ is a real-valued diagonal matrix. We let $\boldsymbol{\lambda} = \text{diag}(\mathbf{\Lambda})$ be a vector containing $\mathbf{A}$ ’s eigenvalues in descending order: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_d$. When $\mathbf{A}$ is PSD, $\lambda_i \geq 0$ for all i . We use the identities $\text{tr}(\mathbf{A}) = \|\boldsymbol{\lambda}\|_1$ and $\|\mathbf{A}\|_F = \|\boldsymbol{\lambda}\|_2$. We let $\mathbf{A}_k = \arg \min_{\mathbf{B}, \text{rank}(\mathbf{B})=k} \|\mathbf{A} - \mathbf{B}\|_F$ denote the optimal k -rank approximation to $\mathbf{A}$. For a PSD matrix $\mathbf{A}$, $\mathbf{A}_k = \mathbf{V}_k \mathbf{\Lambda}_k \mathbf{V}_k^T$, where $\mathbf{V}_k \in \mathbb{R}^{d \times k}$ contains the first k columns of $\mathbf{V}$ and $\mathbf{\Lambda}_k$ is the $k \times k$ top left submatrix of $\mathbf{\Lambda}$.

We state a few results for non-PSD matrices which depend on the nuclear norm. Consider a general square matrix $\mathbf{A} \in \mathbb{R}^{d \times d}$ with singular value decomposition $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$, where $\mathbf{V}, \mathbf{U} \in \mathbb{R}^{d \times d}$ is orthogonal and $\mathbf{\Sigma}$ is a positive diagonal matrix containing $\mathbf{A}$ ’s singular values, $\sigma_1, \dots, \sigma_d$. The nuclear norm $\|\mathbf{A}\|_*$ is equal to $\|\mathbf{A}\|_* = \sum_{i=1}^d \sigma_i$. For PSD $\mathbf{A}$, $\|\mathbf{A}\|_* = \text{tr}(\mathbf{A})$.

Hutchinson’s Analysis. We require a standard bound on the accuracy of Hutchinson’s estimator:

Lemma 2. *Let $\mathbf{A} \in \mathbb{R}^{d \times d}$, $\delta \in (0, 1/2]$, $\ell \in \mathbb{N}$. Let $\text{H}_\ell(\mathbf{A})$ be the ℓ -query Hutchinson estimator defined in (1), implemented with mean 0, i.i.d. sub-Gaussian random variables with constant sub-Gaussian parameter. For fixed constants c, C , if $\ell > c \log(1/\delta)$, then with probability $\geq 1 - \delta$,*

$$|\text{H}_\ell(\mathbf{A}) - \text{tr}(\mathbf{A})| \leq C \sqrt{\frac{\log(1/\delta)}{\ell}} \|\mathbf{A}\|_F.$$

So, if $\ell = O\left(\frac{\log(1/\delta)}{\varepsilon^2}\right)$ then, with probability $\geq 1 - \delta$, $|\text{H}_\ell(\mathbf{A}) - \text{tr}(\mathbf{A})| \leq \varepsilon \|\mathbf{A}\|_F$.

We refer the reader to [RV⁺13] for a formal definition of sub-Gaussian random variables: both normal $\mathcal{N}(0, 1)$ random variables and ± 1 random variables are sub-Gaussian with constant parameter. Lemma 2 is proven in Appendix A for completeness. It is slightly more general than prior work [RA15] in that it applies to non-PSD, and even asymmetric matrices, which will be important in the analysis of our non-adaptive algorithm. A similar result was recently shown in [CK20].

3 Complexity Analysis

We start by providing the technical intuition behind Hutch++. First note that, for a PSD matrix with eigenvalues $\boldsymbol{\lambda}$, $\|\mathbf{A}\|_F \leq \text{tr}(\mathbf{A})$, so Lemma 2 immediately implies that Hutchinson’s estimator

obtains a relative error guarantee with $O(1/\varepsilon^2)$ queries. However, this bound is only tight when $\|\lambda\|_2 \approx \|\lambda\|_1$, i.e., when $\mathbf{A}$ has *significant mass concentrated on just a small number of eigenvalues*.

Hutch++ simply eliminates this possibility by approximately projecting off $\mathbf{A}$'s large eigenvalues using a projection $\mathbf{Q}\mathbf{Q}^T$. By doing so, it only needs to compute a stochastic estimate for the trace of $(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\mathbf{A}(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)$. The error of this estimate is proportional to $\|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\mathbf{A}(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\|_F$, which we show is *always much smaller than* $\text{tr}(\mathbf{A})$. In particular, suppose that $\mathbf{Q} = \mathbf{V}_k$ exactly spanned the top k eigenvectors $\mathbf{A}$ and thus $(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\mathbf{A}(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T) = \mathbf{A} - \mathbf{A}_k$. Then we have:

Lemma 3. *For any PSD matrix $\mathbf{A}$, $\|\mathbf{A} - \mathbf{A}_k\|_F \leq \frac{1}{\sqrt{k}} \text{tr}(\mathbf{A})$.*

Proof. We have $\lambda_{k+1} \leq \frac{1}{k} \sum_{i=1}^k \lambda_i \leq \frac{1}{k} \text{tr}(\mathbf{A})$, so:

$$\|\mathbf{A} - \mathbf{A}_k\|_F^2 = \sum_{i=k+1}^d \lambda_i^2 \leq \lambda_{k+1} \sum_{i=k+1}^d \lambda_i \leq \frac{1}{k} \text{tr}(\mathbf{A}) \sum_{i=k+1}^d \lambda_i \leq \frac{1}{k} \text{tr}(\mathbf{A})^2. \quad \square$$

The above analysis can be tightened by a factor of two via Lemma 7 in [GSTV07].

Lemma 3 immediately suggests the possibility of an algorithm with $O(1/\varepsilon)$ query complexity: Set $k = O(1/\varepsilon)$ and split $\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{A}_k) + \text{tr}(\mathbf{A} - \mathbf{A}_k)$. The first term can be computed exactly with $O(1/\varepsilon)$ matrix-vector multiplication queries if $\mathbf{V}_k$ is known, since $\text{tr}(\mathbf{A}_k) = \text{tr}(\mathbf{V}_k^T \mathbf{A} \mathbf{V}_k)$. By **Lemma 3** combined with **Lemma 2**, the second can be estimated to error $\pm \varepsilon \text{tr}(\mathbf{A})$ using just $O(1/\varepsilon)$ queries instead of $O(1/\varepsilon^2)$. Of course, we can't compute $\mathbf{V}_k$ exactly with a small number of matrix-vector multiplication queries, but this is easily resolved by using an approximate projection. Using standard tools from randomized linear algebra, $O(k)$ queries suffices to find a $\mathbf{Q}$ with $\|(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\mathbf{A}(\mathbf{I} - \mathbf{Q}\mathbf{Q}^T)\|_F \leq O(\|\mathbf{A} - \mathbf{A}_k\|_F)$, which is all that is needed for a $O(1/\varepsilon)$ query result.

Concretely, we use **Lemma 3** to prove the following general theorem, from which **Theorem 1** and our non-adaptive algorithmic result will follow as direct corollaries.

Theorem 4. *Let $\mathbf{A} \in \mathbb{R}^{d \times d}$ be PSD, $\delta \in (0, 1/2)$, $\ell \in \mathbb{N}$, $k \in \mathbb{N}$. Let $\tilde{\mathbf{A}}$ and Δ be any matrices with:*

$$\text{tr}(\mathbf{A}) = \text{tr}(\tilde{\mathbf{A}}) + \text{tr}(\Delta) \quad \text{and} \quad \|\Delta\|_F \leq 2\|\mathbf{A} - \mathbf{A}_k\|_F.$$

For fixed constants c, C , if $\ell > c \log(1/\delta)$, then with probability $1 - \delta$, $Z = \left[\text{tr}(\tilde{\mathbf{A}}) + \text{H}_\ell(\Delta) \right]$ satisfies:

$$\left| Z - \text{tr}(\mathbf{A}) \right| \leq 2C \sqrt{\frac{\log(1/\delta)}{k\ell}} \cdot \text{tr}(\mathbf{A}).$$

In particular, if $k = \ell = O\left(\frac{\sqrt{\log(1/\delta)}}{\varepsilon} + \log(1/\delta)\right)$, Z is a $(1 \pm \varepsilon)$ error approximation to $\text{tr}(\mathbf{A})$.

Proof. We have with probability $\geq 1 - \delta$:

$$\begin{aligned} |Z - \text{tr}(\mathbf{A})| &= |\text{H}_\ell(\Delta) - \text{tr}(\Delta)| \quad (\text{since } Z = \text{tr}(\tilde{\mathbf{A}}) + \text{H}_\ell(\Delta) \text{ and } \text{tr}(\mathbf{A}) = \text{tr}(\tilde{\mathbf{A}}) + \text{tr}(\Delta)) \\ &\leq C \sqrt{\frac{\log(1/\delta)}{\ell}} \|\Delta\|_F \quad (\text{by the standard Hutchinson's analysis, Lemma 2}) \\ &\leq 2C \sqrt{\frac{\log(1/\delta)}{\ell}} \|\mathbf{A} - \mathbf{A}_k\|_F \quad (\text{by the assumption that } \|\Delta\|_F \leq 2\|\mathbf{A} - \mathbf{A}_k\|_F) \\ &\leq 2C \sqrt{\frac{\log(1/\delta)}{k\ell}} \text{tr}(\mathbf{A}). \quad (\text{by Lemma 3}) \end{aligned}$$

□

As discussed, [Theorem 4](#) would immediately yield an $O(1/\varepsilon)$ query algorithm if we knew an optimal k -rank approximation for $\mathbf{A}$. Since computing one is infeasible, our first version of Hutch++ ([Algorithm 1](#)) instead uses a projection onto a subspace $\mathbf{Q}$ which is computed with one iteration of the power method. We have:

Theorem 1 Restated. *If [Algorithm 1](#) is implemented with $m = O(\sqrt{\log(1/\delta)}/\varepsilon + \log(1/\delta))$ matrix-vector multiplication queries, then for any PSD $\mathbf{A}$, with probability $\geq 1 - \delta$, the output $\text{Hutch}^{++}(\mathbf{A})$ satisfies: $(1 - \varepsilon) \text{tr}(\mathbf{A}) \leq \text{Hutch}^{++}(\mathbf{A}) \leq (1 + \varepsilon) \text{tr}(\mathbf{A})$.*

Proof. Let $\mathbf{S}$, $\mathbf{G}$, and $\mathbf{Q}$ be as in [Algorithm 1](#). We instantiate [Theorem 4](#) with $\tilde{\mathbf{A}} = \mathbf{Q}^T \mathbf{A} \mathbf{Q}$ and $\Delta = (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T)$. Note that, since $\mathbf{Q}$ is orthogonal, $(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T)$ is a projection matrix, so $(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) = (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T)^2$. This fact, along with the cyclic property of the trace, gives:

$$\text{tr}(\tilde{\mathbf{A}}) = \text{tr}(\mathbf{A} \mathbf{Q} \mathbf{Q}^T) \quad \text{and} \quad \text{tr}(\Delta) = \text{tr}(\mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T)),$$

and thus $\text{tr}(\tilde{\mathbf{A}}) + \text{tr}(\Delta) = \text{tr}(\mathbf{A})$ as required by [Theorem 4](#). Furthermore, since multiplying by a projection matrix can only decrease Frobenius norm, $\|\Delta\|_F^2 \leq \|\mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T)\|_F^2 = \|\mathbf{A} - \mathbf{A} \mathbf{Q} \mathbf{Q}^T\|_F^2$.

Recall that $\mathbf{Q}$ is an orthogonal basis for the column span of $\mathbf{A} \mathbf{S}$, where $\mathbf{S}$ is a random sign matrix with $\frac{m}{3}$ columns. $\mathbf{Q}$ is thus an orthogonal basis for a linear sketch of $\mathbf{A}$'s column space, and it is well known that $\mathbf{Q}$ will align with large eigenvectors of $\mathbf{A}$, and $\|\mathbf{A} - \mathbf{A} \mathbf{Q} \mathbf{Q}^T\|_F^2$ will be small [Sar06, Woo14]. Concretely, applying Corollary 7 and Claim 1 from [MM20], we have that, as long as $\frac{m}{3} \geq O(k + \log(1/\delta))$, with probability $\geq 1 - \delta$:

$$\|\mathbf{A} - \mathbf{A} \mathbf{Q} \mathbf{Q}^T\|_F^2 \leq 2 \|\mathbf{A} - \mathbf{A}_k\|_F^2.$$

Accordingly, $\|\Delta\|_F \leq 2 \|\mathbf{A} - \mathbf{A}_k\|_F^2$ as required by [Theorem 4](#). The result then immediately follows by setting $k = O(\sqrt{\log(1/\delta)}/\varepsilon + \log(1/\delta))$ and noting that $\text{Hutch}^{++}(\mathbf{A}) = \left[\text{tr}(\tilde{\mathbf{A}}) + \text{H}_\ell(\Delta) \right]$ where $\ell = O(\sqrt{\log(1/\delta)}/\varepsilon + \log(1/\delta))$. $\square$

Notably, none of the analysis above uses the fact that $\mathbf{A}$ is PSD except for [Lemma 3](#). However, [Lemma 3](#) holds for any matrix by replacing the trace with the nuclear norm (the two are equal for PSD matrices). So, the following result holds for general square matrices:

Theorem 5. *If [Algorithm 1](#) is implemented with $m = O(\sqrt{\log(1/\delta)}/\varepsilon + \log(1/\delta))$ matrix-vector multiplication queries, then for any $\mathbf{A}$, with probability $\geq 1 - \delta$, the output $\text{Hutch}^{++}(\mathbf{A})$ satisfies:*

$$|\text{Hutch}^{++}(\mathbf{A}) - \text{tr}(\mathbf{A})| \leq \sqrt{\varepsilon} \|\mathbf{A} - \mathbf{A}_{1/\varepsilon}\|_F \leq \varepsilon \|\mathbf{A}\|_*.$$

Using the same number of queries, Hutchinson's estimator achieves a bound of $O(\sqrt{\varepsilon}) \cdot \|\mathbf{A}\|_F$. Since $\|\mathbf{A} - \mathbf{A}_{1/\varepsilon}\|_F \leq \|\mathbf{A}\|_F$, the first inequality in [Theorem 5](#) shows that Hutch++ is never asymptotically slower than Hutchinson's, even for non-PSD matrices. Furthermore, if $\mathbf{A}$ has quickly decaying eigenvalues, this inequality shows that Hutch++ will converge especially quickly. The second inequality mirrors [Theorem 1](#), stating the deviation of Hutch++ in terms of nuclear norm instead of trace. For PSD matrices, the two are equivalent.

3.1 A Non-Adaptive Variant of Hutch++

As discussed in [Section 1](#), [Algorithm 1](#) is *adaptive*: it uses the result of computing $\mathbf{A} \mathbf{S}$ to compute $\mathbf{Q}$, which is then multiplied by $\mathbf{A}$ to compute the $\text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q})$ term. Meanwhile, Hutchinson's estimator is *non-adaptive*: it samples a single random matrix upfront, batch-multiplies by $\mathbf{A}$ once, and computes an approximation to $\text{tr}(\mathbf{A})$ from the result, without any further queries.

Not only is non-adaptivity an interesting theoretical property, but it can be practically useful, since parallelism or block iterative methods often make it faster to multiply an implicit matrix by many vectors at once. With these considerations in mind, we describe a non-adaptive variant of Hutch++, which we call NA-Hutch++. NA-Hutch++ obtains nearly the same theoretical guarantees as [Algorithm 1](#), although it tends to perform slightly worse in our experiments.

We leverage a streaming low-rank approximation result of Clarkson and Woodruff [CW09] which shows that if $\mathbf{S} \in \mathbb{R}^{d \times m}$ and $\mathbf{R} \in \mathbb{R}^{d \times cm}$ are sub-Gaussian random matrices with $m = O(k \log(1/\delta))$ and $c > 1$ a fixed constant, then with probability $1 - \delta$, the matrix $\tilde{\mathbf{A}} = \mathbf{A}\mathbf{R}(\mathbf{S}^T \mathbf{A}\mathbf{R})^+ (\mathbf{A}\mathbf{S})^T$ satisfies $\|\mathbf{A} - \tilde{\mathbf{A}}\|_F \leq 2\|\mathbf{A} - \mathbf{A}_k\|_F$. Here $^+$ denotes the Moore-Penrose pseudoinverse. We can compute $\text{tr}(\tilde{\mathbf{A}})$ efficiently without explicitly constructing $\tilde{\mathbf{A}} \in \mathbb{R}^{d \times d}$ by noting that it is equal to $\text{tr}((\mathbf{S}^T \mathbf{A}\mathbf{R})^+ (\mathbf{A}\mathbf{S})^T (\mathbf{A}\mathbf{R}))$ via the cyclic property of the trace. This yields:

Algorithm 2 NA-Hutch++ (Non-Adaptive variant of Hutch++)

input: Matrix-vector multiplication oracle for matrix, $\mathbf{A} \in \mathbb{R}^{d \times d}$. Number of queries, m .

output: Approximation to $\text{tr}(\mathbf{A})$.

- 1: Fix constants c_1, c_2, c_3 such that $c_1 < c_2$ and $c_1 + c_2 + c_3 = 1$.
 - 2: Sample $\mathbf{S} \in \mathbb{R}^{d \times c_1 m}$, $\mathbf{R} \in \mathbb{R}^{d \times c_2 m}$, and $\mathbf{G} \in \mathbb{R}^{d \times c_3 m}$ with i.i.d. $\{+1, -1\}$ entries.
 - 3: Compute $\mathbf{Z} = \mathbf{A}\mathbf{R}$ and $\mathbf{W} = \mathbf{A}\mathbf{S}$.
 - 4: **return** $\text{NA-Hutch++}(\mathbf{A}) = \text{tr}((\mathbf{S}^T \mathbf{Z})^+ (\mathbf{W}^T \mathbf{Z})) + \frac{1}{c_3 m} [\text{tr}(\mathbf{G}^T \mathbf{A}\mathbf{G}) - \text{tr}(\mathbf{G}^T \mathbf{Z}(\mathbf{S}^T \mathbf{Z})^+ \mathbf{W}^T \mathbf{G})]$
-

NA-Hutch++ requires m matrix-vector multiplications with $\mathbf{A}$. In our experiments, it works well with $c_1 = c_3 = 1/4$ and $c_2 = 1/2$. Assuming $m < d$, it requires $O(dm^2)$ further runtime, to perform the matrix multiplications on line 4 and to compute $(\mathbf{S}^T \mathbf{Z})^+$, which takes $O(dm^2 + m^3)$ time.

Theorem 6. *If NA-Hutch++ is implemented with $m = O(\log(1/\delta)/\varepsilon)$ matrix-vector multiplication queries and $\frac{c_2}{c_1}$ a sufficiently large constant, then for any PSD $\mathbf{A}$, with probability $\geq 1 - \delta$, the output $\text{NA-Hutch++}(\mathbf{A})$ satisfies: $(1 - \varepsilon) \text{tr}(\mathbf{A}) \leq \text{NA-Hutch++}(\mathbf{A}) \leq (1 + \varepsilon) \text{tr}(\mathbf{A})$.*

Proof. We apply [Theorem 4](#) with $\tilde{\mathbf{A}} = \mathbf{Z}(\mathbf{S}^T \mathbf{Z})^+ \mathbf{W}^T$, $\Delta = \mathbf{A} - \tilde{\mathbf{A}}$, $k = O(1/\varepsilon)$ and $\ell = c_3 m = O(\frac{\log(1/\delta)}{\varepsilon})$. $\text{tr}(\mathbf{A}) = \text{tr}(\tilde{\mathbf{A}}) + \text{tr}(\Delta)$ and $\text{NA-Hutch++}(\mathbf{A}) = [\text{tr}(\tilde{\mathbf{A}}) + \text{H}_\ell(\Delta)]$. By Theorem 4.7 of [CW09], since $c_1 m = O(k \log(1/\delta))$, $\|\Delta\|_F \leq 2\|\mathbf{A} - \mathbf{A}_k\|_F$ with probability $\geq 1 - \delta$ as required. $\square$

4 Lower Bounds

A natural question is if the $O(1/\varepsilon)$ matrix-vector query bound of [Theorem 1](#) and [Theorem 6](#) is tight. In this section, we prove that it is up to a logarithmic factor, even for algorithms that perform *adaptive* queries like Hutch++. Our lower bound is via a reduction to communication complexity: we show that a better algorithm for PSD trace estimation would imply a better 2-party communication protocol for the Gap-Hamming problem, which would violate known adaptive lower bounds for that problem [CR12]. To prove this result we need to assume a fixed precision model of computation. Specifically we require that the entries in each query vector $\mathbf{r}$ are integers bounded in absolute value by 2^b , for some fixed constant b . By scaling, this captures the setting where the query vectors are non-integer, but have bounded precision. Formally, we prove in [Section 4.1](#):

Theorem 7. *Any algorithm that accesses a positive semidefinite matrix $\mathbf{A}$ via matrix-vector multiplication queries $\mathbf{A}\mathbf{r}_1, \dots, \mathbf{A}\mathbf{r}_m$, where $\mathbf{r}_1, \dots, \mathbf{r}_m$ are possibly adaptively chosen vectors with integer entries in $\{-2^b, \dots, 2^b\}$, requires $m = \Omega\left(\frac{1}{\varepsilon(b + \log(1/\varepsilon))}\right)$ such queries to output an estimate t so that, with probability $> 2/3$, $(1 - \varepsilon) \text{tr}(\mathbf{A}) \leq t \leq (1 + \varepsilon) \text{tr}(\mathbf{A})$.*

For constant b our lower bound is $\Omega\left(\frac{1}{\varepsilon \log(1/\varepsilon)}\right)$, which matches [Theorem 1](#) and [Theorem 6](#) up to a $\log(1/\varepsilon)$ factor. We also provide an alternative lower bound which holds in the real RAM model of computation (all inputs and arithmetic operations involve real numbers). This second lower bound is tight up to constants, but only applies to non-adaptive algorithms. It is proven using different information theoretic techniques – we reduce to a hypothesis testing problem involving *negatively spiked* covariance matrices [CMW15, PWBM18]. Formally, we prove in [Appendix B](#):

Theorem 8. *Any algorithm that accesses a positive semidefinite matrix $\mathbf{A}$ through matrix-vector multiplication queries $\mathbf{A}\mathbf{r}_1, \dots, \mathbf{A}\mathbf{r}_m$, where $\mathbf{r}_1, \dots, \mathbf{r}_m$ are real valued non-adaptively chosen vectors requires $m = \Omega\left(\frac{1}{\varepsilon}\right)$ such queries to output an estimate t so that, with probability $> 3/4$, $(1 - \varepsilon) \text{tr}(\mathbf{A}) \leq t \leq (1 + \varepsilon) \text{tr}(\mathbf{A})$.*

4.1 Adaptive lower bound

The proof of [Theorem 7](#) is based on reducing the Gap-Hamming problem to trace estimation. This problem has been well studied in communication complexity since its introduction in [IW03].

Problem 1 (Gap-Hamming). *Let Alice and Bob be communicating parties who hold vectors $\mathbf{s} \in \{-1, 1\}^n$ and $\mathbf{t} \in \{-1, 1\}^n$, respectively. The Gap-Hamming problem asks Alice and Bob to return:*

$$1 \text{ if } \langle \mathbf{s}, \mathbf{t} \rangle \geq \sqrt{n} \quad \text{and} \quad -1 \text{ if } \langle \mathbf{s}, \mathbf{t} \rangle \leq -\sqrt{n}.$$

A tight lower bound on the unbounded round, randomized communication complexity of this problem was first proven in [CR12], with alternative proofs appearing in [Vid12, She12]. Formally:

Lemma 9 (Theorem 2.6 in [CR12]). *The randomized communication complexity for solving [Problem 1](#) with probability $\geq 2/3$ is $\Omega(n)$ bits.*

With [Lemma 9](#) in place, we have all we need to prove [Theorem 7](#).

Proof of Theorem 7. Fix a perfect square $n \in \mathbb{N}$. Consider an instance of [Problem 1](#) with inputs $\mathbf{s} \in \mathbb{R}^n$ and $\mathbf{t} \in \mathbb{R}^n$. Let $\mathbf{S} \in \mathbb{R}^{\sqrt{n} \times \sqrt{n}}$ and $\mathbf{T} \in \mathbb{R}^{\sqrt{n} \times \sqrt{n}}$ contain the entries of $\mathbf{s}$ and $\mathbf{t}$ rearranged into matrices (e.g., placed left-to-right, top-to-bottom). Let $\mathbf{Z} = \mathbf{S} + \mathbf{T}$ and let $\mathbf{A} = \mathbf{Z}^T \mathbf{Z}$. $\mathbf{A}$ is positive semidefinite and we have:

$$\text{tr}(\mathbf{A}) = \|\mathbf{Z}\|_F^2 = \|\mathbf{s} + \mathbf{t}\|_2^2 = \|\mathbf{s}\|_2^2 + \|\mathbf{t}\|_2^2 + 2\langle \mathbf{s}, \mathbf{t} \rangle = 2n + 2\langle \mathbf{s}, \mathbf{t} \rangle.$$

If $\langle \mathbf{s}, \mathbf{t} \rangle \geq \sqrt{n}$ then we will have $\text{tr}(\mathbf{A}) \geq 2(n + \sqrt{n})$ and if $\langle \mathbf{s}, \mathbf{t} \rangle \leq -\sqrt{n}$ then we will have $\text{tr}(\mathbf{A}) \leq 2(n - \sqrt{n})$. So, if Alice and Bob can approximate $\text{tr}(\mathbf{A})$ up to relative error $(1 \pm 1/\sqrt{n})$, then they can solve [Problem 1](#). We claim that they can do so with just $O(m \cdot \sqrt{n}(\log n + b))$ bits of communication if there exists an m -query adaptive matrix-vector multiplication algorithm for positive semidefinite trace estimation achieving error $(1 \pm 1/\sqrt{n})$.

Specifically, Alice takes charge of running the query algorithm. To compute $\mathbf{A}\mathbf{r}$ for a vector $\mathbf{r}$, Alice and Bob first need to compute $\mathbf{Z}\mathbf{r}$. To do so, Alice sends $\mathbf{r}$ to Bob, which takes $O(\sqrt{n} \cdot b)$ bits since $\mathbf{r}$ has entries bounded by 2^b . Bob then computes $\mathbf{T}\mathbf{r}$, which has entries bounded by $\sqrt{n}2^b$. He sends the result to Alice, using $O(\sqrt{n}(b + \log n))$ bits. Upon receiving $\mathbf{T}\mathbf{r}$, Alice computes $\mathbf{Z}\mathbf{r} = \mathbf{S}\mathbf{r} + \mathbf{T}\mathbf{r}$. Next, they need to multiply $\mathbf{Z}\mathbf{r}$ by $\mathbf{Z}^T$ to obtain $\mathbf{A}\mathbf{r} = \mathbf{Z}^T \mathbf{Z}\mathbf{r}$. To do so, Alice sends $\mathbf{Z}\mathbf{r}$ to Bob (again using $O(\sqrt{n}(b + \log n))$ bits) who computes $\mathbf{T}^T \mathbf{Z}\mathbf{r}$. The entries in this vector are bounded by $2n2^b$, so Bob sends the result back to Alice using $O(\sqrt{n}(b + \log n))$ bits. Finally, Alice computes $\mathbf{S}^T \mathbf{Z}\mathbf{r}$ and adds the result to $\mathbf{T}^T \mathbf{Z}\mathbf{r}$ to obtain $\mathbf{Z}^T \mathbf{Z}\mathbf{r} = \mathbf{A}\mathbf{r}$. Given this result, Alice chooses the next query vector according to the algorithm and repeats.

Overall, running the full matrix-vector query algorithm requires $O(m \cdot \sqrt{n}(\log n + b))$ bits of communication. So, from [Lemma 9](#) we have that $m = \Omega(\sqrt{n}/(\log n + b))$ queries are needed to approximate the trace to accuracy $1 \pm \varepsilon$ for $\varepsilon = 1/\sqrt{n}$, with probability $> 2/3$. $\square$

5 Variance Analysis

In this section, we bound the variance of a version of the Hutch++ estimator ([Algorithm 3](#)), which involves Gaussian random vectors. While the high-probability bounds of [Theorem 1](#) and [Theorem 5](#) hold for this version of the algorithm, the variance bounds have the advantage of involving (small) explicit constants. They be used to obtain high probability bounds with similarly explicit constants via Chebyshev's inequality, albeit with a worse δ dependence than [Theorem 1](#) and [Theorem 5](#).

Algorithm 3 Gaussian-Hutch++ (Gaussian Variant of Hutch++)

input: Matrix-vector multiplication oracle for matrix, $\mathbf{A} \in \mathbb{R}^{d \times d}$. Number of queries, m .

output: Approximation to $\text{tr}(\mathbf{A})$.

- 1: Sample $\mathbf{S} \in \mathbb{R}^{d \times \frac{m+2}{4}}$ with i.i.d. $\mathcal{N}(0, 1)$ entries and $\mathbf{G} \in \mathbb{R}^{d \times \frac{m-2}{2}}$ with i.i.d. $\{+1, -1\}$ entries.
 - 2: Compute an orthonormal basis $\mathbf{Q} \in \mathbb{R}^{d \times \frac{m+2}{4}}$ for the span of $\mathbf{AS}$ (e.g., via QR decomposition).
 - 3: **return** Gaussian-Hutch++($\mathbf{A}$) = $\text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + \frac{2}{m-2} \text{tr}(\mathbf{G}^T (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{G})$.
-

The only difference between [Algorithm 3](#) and [Algorithm 1](#) is that $\mathbf{S}$ is now Gaussian, and constants are set slightly differently (to minimize variance). Our main result follows:

Theorem 10. *If [Algorithm 3](#) is implemented with m queries, then for PSD $\mathbf{A}$,*

$$\mathbb{E}[\text{Gaussian-Hutch++}(\mathbf{A})] = \text{tr}(\mathbf{A}) \quad \text{and} \quad \text{Var}[\text{Gaussian-Hutch++}(\mathbf{A})] \leq \frac{16}{(m-2)^2} \text{tr}^2(\mathbf{A})$$

Before stating the proof, we import three theorems. The first is on the variance of Hutchinson's estimator implemented with Gaussians, and is easy to derive directly:

Imported Lemma 11 (Lemma 1 from [AT11]). *Hutchinson's estimator implemented with Gaussian random entries has $\mathbb{E}[H_\ell(\mathbf{A})] = \text{tr}(\mathbf{A})$ and $\text{Var}[H_\ell(\mathbf{A})] = \frac{2}{\ell} \|\mathbf{A}\|_F^2$.*

We also require a result on the *expected error* of a randomized low-rank approximation. In contrast, the proof of [Theorem 1](#) uses a *high-probability* result. Note that the following result is why we use Gaussian random vectors instead of random sign bits.

Imported Theorem 12 (Theorem 10.5 from [HMT11]). *Fix target rank $k \geq 2$ and oversampling parameter $p \geq 2$. Let $\mathbf{S} \in \mathbb{R}^{d \times (k+p)}$ with i.i.d. $\mathcal{N}(0, 1)$ entries, and let $\mathbf{Q} \in \mathbb{R}^{d \times (k+p)}$ be an orthonormal span the columns of $\mathbf{AS}$. Then,*

$$\mathbb{E} [\|(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A}\|_F^2] \leq (1 + \frac{k}{p-1}) \|\mathbf{A} - \mathbf{A}_k\|_F^2$$

Finally, we state a strengthening of [Lemma 3](#).

Imported Lemma 13 (Lemma 7 from [GSTV07]). *Let $\mathbf{A}_k$ be the best rank- k approximation to PSD matrix $\mathbf{A}$. Then $\|\mathbf{A} - \mathbf{A}_k\|_F \leq \frac{1}{2\sqrt{k}} \text{tr}(\mathbf{A})$.*

Proof of Theorem 10. Let q be the number of columns in $\mathbf{S}$ and ℓ be the number of columns in $\mathbf{G}$, constants that will be chosen shortly. Note that Algorithm 3 uses $m = 2q + \ell$ matrix vector multiplications. For notational simplicity, let $\tilde{\text{tr}}(\mathbf{A}) := \text{Gaussian-Hutch}^{++}(\mathbf{A})$. We have $\tilde{\text{tr}}(\mathbf{A}) = \text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + H_\ell((\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T))$.

We first prove the unbiased expectation. Note that it suffices to prove that, for any fixed $\mathbf{Q}$, $\mathbb{E}[\tilde{\text{tr}}(\mathbf{A})|\mathbf{Q}] = \text{tr}(\mathbf{A})$. This follows from cyclic property of trace, the fact that $\mathbf{I} - \mathbf{Q} \mathbf{Q}^T$ is idempotent, and Imported Lemma 11:

$$\begin{aligned} \mathbb{E}[\tilde{\text{tr}}(\mathbf{A})|\mathbf{Q}] &= \mathbb{E}[\text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + H_\ell((\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T))|\mathbf{Q}] \\ &= \text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + \mathbb{E}[H_\ell((\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T))|\mathbf{Q}] \\ &= \text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + \text{tr}(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \\ &= \text{tr}(\mathbf{A}) \end{aligned}$$

Then, to bound the variance, we appeal to the Law of Total Variance:

$$\text{Var}[\tilde{\text{tr}}(\mathbf{A})] = \mathbb{E}[\text{Var}[\tilde{\text{tr}}(\mathbf{A})|\mathbf{Q}]] + \text{Var}[\mathbb{E}[\tilde{\text{tr}}(\mathbf{A})|\mathbf{Q}]] \quad (2)$$

The second term is always zero because, as shown above, we always have that $\mathbb{E}[\tilde{\text{tr}}(\mathbf{A})|\mathbf{Q}] = \text{tr}(\mathbf{A})$. So, we only have to bound the first term in Equation 2:

$$\begin{aligned} \text{Var}[\tilde{\text{tr}}(\mathbf{A})] &= \mathbb{E}[\text{Var}[\tilde{\text{tr}}(\mathbf{A})|\mathbf{Q}]] = \mathbb{E}[\text{Var}[\text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q}) + H_\ell((\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T))|\mathbf{Q}]] \\ &= \mathbb{E}[\text{Var}[H_\ell((\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T))|\mathbf{Q}]] \\ &= \mathbb{E}\left[\frac{2}{\ell} \|(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A} (\mathbf{I} - \mathbf{Q} \mathbf{Q}^T)\|_F^2\right] \quad (\text{Imported Lemma 11}) \\ &\leq \frac{2}{\ell} \mathbb{E}[\|(\mathbf{I} - \mathbf{Q} \mathbf{Q}^T) \mathbf{A}\|_F^2] \quad (\text{submultiplicativity}) \\ &\leq \frac{2}{\ell} \left(1 + \frac{k}{p-1}\right) \|\mathbf{A} - \mathbf{A}_k\|_F^2 \quad (\text{Imported Theorem 12}) \\ &\leq \frac{2}{\ell} \left(1 + \frac{k}{p-1}\right) \frac{1}{4k} \text{tr}^2(\mathbf{A}). \quad (\text{Imported Lemma 13}) \end{aligned}$$

Following Imported Theorem 12, $k, p \geq 2$ are any values satisfying $q = k + p$, and the bound is minimized when $p - 1 = k$. This yields a variance bound of $\frac{1}{\ell k} \text{tr}^2(\mathbf{A})$. Under the constraint $m = 2q + \ell$, where $q = 2k + 1$, $\frac{1}{\ell k}$ is minimized by setting $k = \frac{m-4}{8}$ and $\ell = m - \frac{m+4}{2}$, which yields a bound of $\frac{1}{\ell k} = \frac{16}{(m-2)^2}$. $\square$

Above, only Imported Lemma 13 uses the fact that $\mathbf{A}$ is PSD. Furthermore, the proof of Imported Lemma 13 in [GSTV07] actually implies that $\|\mathbf{A} - \mathbf{A}_k\|_F \leq \frac{1}{2\sqrt{k}} \|\mathbf{A}\|_*$ for any $\mathbf{A}$, where $\|\cdot\|_*$ is the nuclear norm. By repeating the above analysis, we have the following:

Lemma 14. For any $\mathbf{A}$, Algorithm 3 has $\mathbb{E}[\text{Gaussian-Hutch}^{++}(\mathbf{A})] = \text{tr}(\mathbf{A})$ as well as

$$\text{Var}[\text{Hutch}^{++}(\mathbf{A})] \leq \frac{8}{m-2} \|\mathbf{A} - \mathbf{A}_k\|_F^2 \leq \frac{16}{(m-2)^2} \|\mathbf{A}\|_*^2$$

where $k = \frac{m-2}{8} - 1$.

Like in Theorem 5, the first inequality shows how the decay of $\mathbf{A}$'s eigenvalues impacts the variance of Gaussian-Hutch⁺⁺, while the second inequality is an analog to the variance guarantee in Theorem 10.

6 Experimental Validation

We complement our theory with experiments on synthetic matrices and real-world trace estimation problems. Code for Hutch++ and NA-Hutch++ is available at <https://github.com/RaphaelArkadyMeyerNYU/HutchPlusPlus>. We compare these methods to four algorithms, including both our adaptive and non-adaptive methods:

- **Hutchinson’s.** The standard estimator run with $\{+1, -1\}$ random vectors.
- **Subspace Projection.** The method from [SAI17], which computes an orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{d \times k}$ that approximately spans the top eigenvector subspace of $\mathbf{A} \in \mathbb{R}^{d \times d}$ and returns $\text{tr}(\mathbf{Q}^T \mathbf{A} \mathbf{Q})$ as an approximation to $\text{tr}(\mathbf{A})$. A similar approach is employed in [LZ21]. [SAI17] computes $\mathbf{Q}$ using subspace iteration, which requires $k(q + 1)$ matrix-vector multiplications when run for q iterations. A larger q results in a more accurate $\mathbf{Q}$, but requires more multiplications. As in [SAI17], we found that setting $q = 1$ gave the best performance, so we did so in our experiments. With $q = 1$, this method is similar to Hutch++, except that it does not approximate the remainder of the trace outside the top eigenspace.
- **Hutch++.** The adaptive method of Algorithm 1 with $\{+1, -1\}$ random vectors.
- **NA-Hutch++.** The non-adaptive method of Algorithm 2 with $c_1 = c_3 = 1/4$ and $c_2 = 1/2$ and $\{+1, -1\}$ random vectors.

6.1 Synthetic Matrices

We first test the methods above on random matrices with power law spectra. For varying constant c , we let $\mathbf{\Lambda}$ be diagonal with $\Lambda_{ii} = i^{-c}$. We generate a random orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{5000 \times 5000}$ by orthogonalizing a random Gaussian matrix and set $\mathbf{A} = \mathbf{Q}^T \mathbf{\Lambda} \mathbf{Q}$. $\mathbf{A}$ ’s eigenvalues are the values in $\mathbf{\Lambda}$. A larger c results in a more quickly decaying spectrum, so we expect Subspace Projection to perform well. A smaller c results in a slowly decaying spectrum, which will mean that $\|\mathbf{A}\|_F \ll \text{tr}(\mathbf{A})$. In this case, we expect Hutchinson’s to outperform its worst case multiplicative error bound: instead of error $\pm \varepsilon \text{tr}(\mathbf{A})$ after $O(1/\varepsilon^2)$ matrix-multiplication queries, Lemma 2 predicts error on the order of $\pm \varepsilon \|\mathbf{A}\|_F$. Concretely, for dimension $d = 5000$ and $c = 2$, we have $\|\mathbf{A}\|_F = .63 \cdot \text{tr}(\mathbf{A})$ and for $c = .5$ we have $\|\mathbf{A}\|_F = .02 \cdot \text{tr}(\mathbf{A})$. In general, unlike the Subspace Projection method and Hutchinson’s estimator, we expect Hutch++ and NA-Hutch++ to be less sensitive to $\mathbf{A}$ ’s spectrum.

In Figure 1 we plot results for various c . Relative error should scale roughly as $\varepsilon = O(m^{-\gamma})$, where $\gamma = 1/2$ for Hutchinson’s and $\gamma = 1$ for Hutch++ and NA-Hutch++. We thus use log-log plots, where we expect a linear relationship between the error ε and number of iterations m .

The superior performance of Hutch++ and NA-Hutch++ shown in Figure 1 is not surprising. These methods are designed to achieve the “best of both worlds”: when $\mathbf{A}$ ’s spectrum decays quickly, our methods approximate $\text{tr}(\mathbf{A})$ well by projecting off the top eigenvalues. When it decays slowly, they perform essentially no worse than Hutchinson’s. We note that the adaptivity of Hutch++ leads to consistently better performance over NA-Hutch++, and the method is simpler to implement as we do not need to set the constants c_1, c_2, c_3 . Accordingly, this is the method we move forward with in our real data experiments.

6.2 Real Matrices

To evaluate the real-world performance of Hutch++ we test it in the common setting where $\mathbf{A} = f(\mathbf{B})$. In most applications, $\mathbf{B}$ is symmetric with eigendecomposition $\mathbf{V}^T \mathbf{\Lambda} \mathbf{V}$, and $f : \mathbb{R} \rightarrow \mathbb{R}$ is a function on real valued inputs. Then we have $f(\mathbf{B}) = \mathbf{V}^T f(\mathbf{\Lambda}) \mathbf{V}$ where $f(\mathbf{\Lambda})$ is simply f applied to the real-valued eigenvalues on the diagonal of $\mathbf{\Lambda}$. When f returns negative values, $\mathbf{A}$ may not be

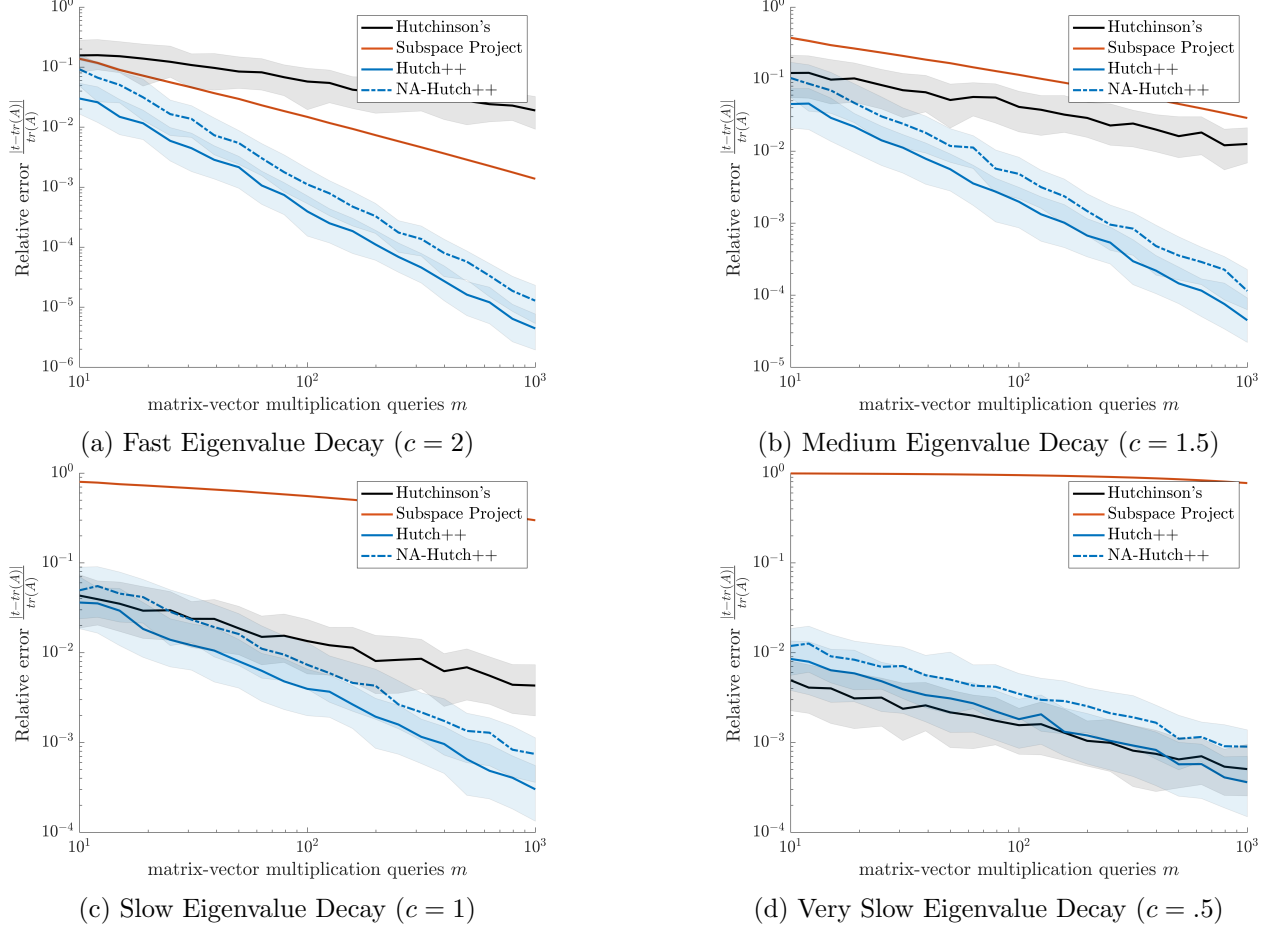


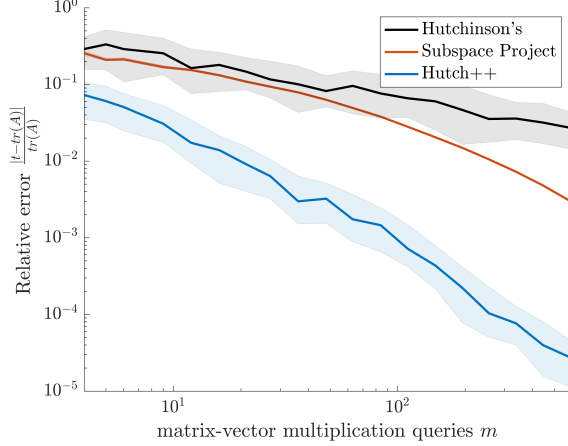
Figure 1: Relative error versus number of matrix-vector multiplication queries for trace approximation algorithms run on random matrices with power law spectra. We report the median relative error of the approximation t after 200 trials. The upper and lower bounds of the shaded region around each curve are the 25th and 75th percentile errors. Subspace Projection has consistently low variance, but as expected, only performs better than Hutchinson’s when $c = 2$ and there is very fast eigenvalue decay. Hutch++ and NA-Hutch++ typically outperform both methods.

postive semidefinite. Generally, computing $f(\mathbf{B})$ explicitly requires a full eigendecomposition and thus $\Omega(n^3)$ time. However, many iterative methods can more quickly approximate matrix-vector queries of the form $\mathbf{A}\mathbf{r} = f(\mathbf{B})\mathbf{r}$. The most popular and general is the Lanczos method, which we employ in our experiments [UCS17, MMS18].²

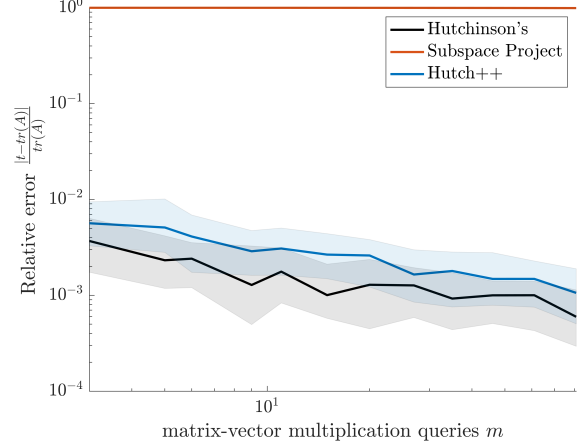
We consider trace estimation in three example applications, involving both PSD and non-PSD matrices. We test on relatively small inputs, for which we can explicitly compute $\text{tr}(f(\mathbf{B}))$ to use as a baseline for the approximation error. However, our methods can scale to much larger matrices.

Graph Estrada Index. Given the binary adjacency matrix $\mathbf{B} \in \{0,1\}^{d \times d}$ of a graph G , the Estrada index is defined as $\text{tr}(\exp(\mathbf{B}))$ [Est00, dIPGR07], where $\exp(x) = e^x$. This index measures the strength of connectivity within G . A simple transformation of the Estrada index yields the *natural connectivity* metric, defined as $\log(\frac{1}{d} \text{tr}(\exp(\mathbf{B})))$ [JBYJHZ10, EHB12].

²We use the implementation of Lanczos available at <https://github.com/cpmusco/fast-pcr>, but modified to block matrix-vector multiplies when run on multiple query vectors.



(a) $\mathbf{A} = \exp(\mathbf{B})$, where $\mathbf{B}$ is the Roget's Thesaurus semantic graph adjacency matrix. For use in Estrada index computation. $\mathbf{A}$ is PSD.



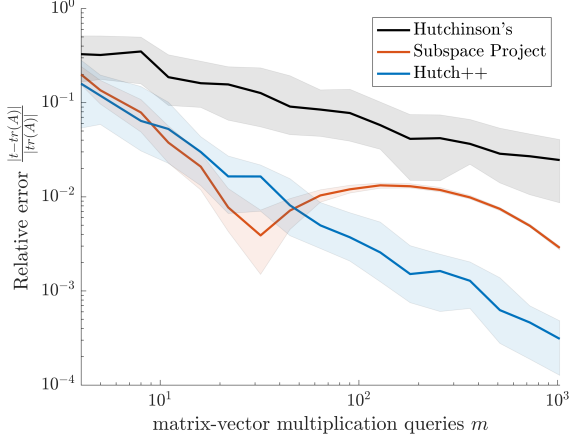
(b) $\mathbf{A} = \log(\mathbf{B} + \lambda \mathbf{I})$, where $\mathbf{B}$ is a 2D Gaussian process kernel covariance matrix. For use in log-likelihood computation. $\mathbf{A}$ is not PSD.

Figure 2: Relative error versus number of matrix-vector multiplication queries for trace approximations of transformed matrices, which were multiplied by vectors using the Lanczos method. We report median relative error of the approximation t after 100 trials. The upper and lower bounds of the shaded region around each curve are the 25th and 75th percentile errors. As expected, Subspace Project and Hutch++ outperform Hutchinson's when $\mathbf{A} = \exp(\mathbf{B})$, as exponentiating leads to a quickly decaying spectrum. On the other hand, Hutchinson's performs well for $\mathbf{A} = \log(\mathbf{B} + \lambda \mathbf{I})$, which has a very flat spectrum. Hutch++ is still essentially as fast. Subspace Project fails in this case because the top eigenvalues of $\mathbf{A}$ do not dominate its trace.

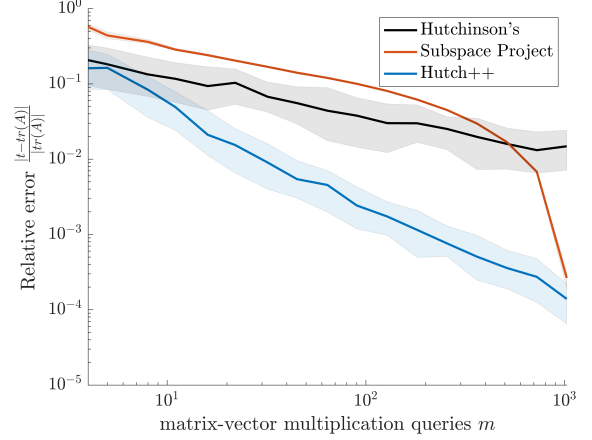
In our experiments, we approximated the Estrada index of the Roget's Thesaurus semantic graph, available from [BM06]. The Estrada index of this 1022 node graph was originally studied in [EH08]. We use the Lanczos method to approximate matrix multiplication with $\exp(\mathbf{B})$, running it for 40 iterations, after which the error of application was negligible compared to the approximation error of trace estimation. Results are shown in Figure 2.

Gaussian Process Log Likelihood. Let $\mathbf{B} \in \mathbb{R}^{d \times d}$ be a PSD kernel covariance matrix and let $\lambda \geq 0$ be a regularization parameter. In Gaussian process regression, the model log likelihood computation requires computing $\log \det(\mathbf{B} + \lambda \mathbf{I}) = \text{tr}(f(\mathbf{B}))$ where $f(x) = \log(x + \lambda)$ [WR96, Ras04]. This quantity must be computed repeatedly for different choices of $\mathbf{B}$ and λ during hyperparameter optimization, and it is often approximated using Hutchinson's method [BDKZ15, UCS17, HMAS17, DEN⁺17]. We note that, while $\mathbf{B}$ is positive semidefinite, $\log(\mathbf{B} + \lambda \mathbf{I})$ typically will not be. So our strongest theoretical bounds do not apply in this case, but Hutch++ can be applied unmodified, and as we see in Figure 2, still gives good performance.

In our experiments we consider a benchmark 2D Gaussian process regression problem from the GIS literature, involving precipitation data from Slovakia [NM13]. $\mathbf{B}$ is the kernel covariance matrix on 6400 randomly selected training points out of 196,104 total points. Following the setup of [EMM20], we let $\mathbf{B}$ be a Gaussian kernel matrix with width parameter $\gamma = 64$ and regularization parameter $\lambda = .008$, both determined via cross-validation on ℓ_2 regression loss.

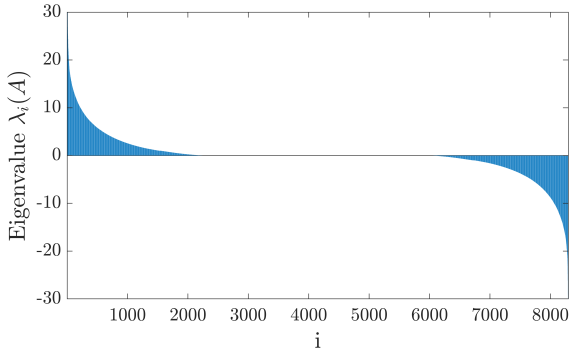


(a) $\mathbf{A} = \mathbf{B}^3$ where $\mathbf{B}$ is a Wikipedia voting network adjacency matrix. For use in triangle counting. $\mathbf{A}$ is not PSD.

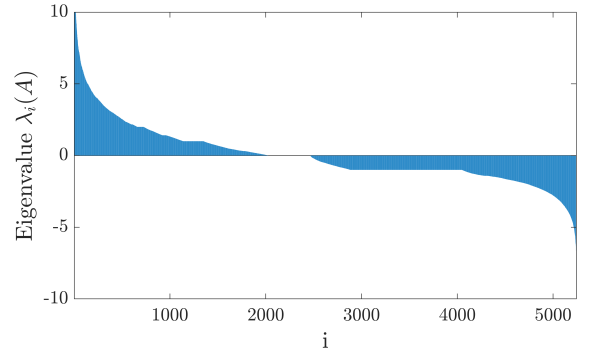


(b) $\mathbf{A} = \mathbf{B}^3$ where $\mathbf{B}$ is an arXiv.org citation network adjacency matrix. For use in triangle counting. $\mathbf{A}$ is not PSD.

Figure 3: Relative error versus number of matrix-vector multiplication queries for trace approximations of transformed matrices. We report the median relative error of the approximation t after 100 trials. The upper and lower bounds of the shaded region around each curve are the 25th and 75th percentile errors. Hutch++ still outperforms the baseline methods even though $\mathbf{A}$ is not PSD. We note that Subspace Project has somewhat uneven performance: increasing m will take into account a larger number of top eigenvalues when approximating the trace. However, since these may be positive or negative, approximation error does not monotonically decrease. Hutch++ is not sensitive to this issue since it does not use *just* the top eigenvalues: see Figure 4 for more discussion.



(a) Eigenvalues for Wikipedia voting network adjacency matrix $\mathbf{B}$.



(b) Eigenvalues for arXiv.org citation network adjacency matrix $\mathbf{B}$.

Figure 4: Even when estimating the trace of a non-PSD matrix like $\mathbf{A} = \mathbf{B}^3$, which for the triangle counting examples above will have both positive and negative eigenvalues, Hutch++ can far outperform Hutchinson's method. As Theorem 5 and Lemma 14 both suggest, it will approximately project off the largest *magnitude* eigenvalues from $\mathbf{A}$ (whether positive or negative), which then reduces the variance of estimating the trace $\text{tr}(\mathbf{A}) = \sum_{i=1}^d \lambda_i(\mathbf{B})^3$.

Graph Triangle Counting. Given the binary adjacency matrix $\mathbf{B} \in \{0, 1\}^{d \times d}$ of an undirected graph G , the number of triangles in G is equal to $\frac{1}{6} \text{tr}(\mathbf{B}^3)$. The triangle count is an important measure of local connectivity and extensive research studies its efficient approximation [SW05,

BBCG08, PT12]. Popular approaches include applying Hutchinson’s method to $\mathbf{A} = \mathbf{B}^3$ [Avr10], or using the EigenTriangle estimator, which is similar to the Subspace Projection method [Tso08].

In our experiments, we study approximate triangle counting on two common benchmark graphs: an arXiv.org collaboration network³ with 5,243 nodes and 48,260 triangles, and a Wikipedia administrator voting network⁴ with 7,115 nodes and 608,389 triangles. We again note that the adjacency matrix $\mathbf{B}$ is not positive semidefinite, and neither is $\mathbf{A} = \mathbf{B}^3$. Nevertheless, we can apply Hutch++ and see very strong performance. In this setting we do not need to apply Lanczos for matrix-vector query computation: $\mathbf{A}\mathbf{r}$ can be computed exactly using three matrix-vector multiplications with $\mathbf{B}$. Results are shown Figure 3 with graph spectral visualized in Figure 4

Acknowledgments

The authors would like to thank Joel A. Tropp for suggesting the variance analysis in Section 5, as well as other valuable comments on the paper. D. Woodruff would like to thank support from the National Institute of Health (NIH) grant 5R01 HG 10798-2 and a Simons Investigator Award.

References

- [Ach03] Dimitris Achlioptas. Database-friendly random projections: Johnson-Lindenstrauss with binary coins. *Journal of Computer and System Sciences*, 66(4):671–687, 2003. Preliminary version in the 20th Symposium on Principles of Database Systems (PODS).
- [APJ⁺18] Ryan P. Adams, Jeffrey Pennington, Matthew J. Johnson, Jamie Smith, Yaniv Ovadia, Brian Patton, and James Saunderson. Estimating the spectral density of large implicit matrices. *arXiv:1802.03451*, 2018.
- [AT11] Haim Avron and Sivan Toledo. Randomized algorithms for estimating the trace of an implicit symmetric positive semi-definite matrix. *Journal of the ACM*, 58(2), 2011.
- [Avr10] Haim Avron. Counting triangles in large graphs using randomized matrix trace estimation. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2010.
- [BBCG08] Luca Becchetti, Paolo Boldi, Carlos Castillo, and Aristides Gionis. Efficient semi-streaming algorithms for local triangle counting in massive graphs. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 16–24, 2008.
- [BDKZ15] Christos Boutsidis, Petros Drineas, Prabhanjan Kambadur, and Anastasios Zouzias. A randomized algorithm for approximating the log determinant of a symmetric positive definite matrix. *Linear Algebra and its Applications*, 533, 03 2015.
- [BHSW20] Mark Braverman, Elad Hazan, Max Simchowitz, and Blake Woodworth. The gradient complexity of linear regression. In *Proceedings of the 33rd Annual Conference on Computational Learning Theory (COLT)*, volume 125, pages 627–647, 2020.

³Link: <https://snap.stanford.edu/data/ca-GrQc.html>.

⁴Link: <https://snap.stanford.edu/data/wiki-Vote.html>.

- [BKKS20] Vladimir Braverman, Robert Krauthgamer, Aditya Krishnan, and Roi Sinoff. Schatten norms in matrix streams: Hello sparsity, goodbye dimension. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [BM06] Vladimir Batagelj and Andrej Mrvar. Pajek datasets. <http://vlado.fmf.uni-lj.si/pub/networks/data/>, 2006.
- [CEM⁺15] Michael Cohen, Sam Elder, Cameron Musco, Christopher Musco, and Madalina Persu. Dimensionality reduction for k -means clustering and low rank approximation. In *Proceedings of the 47th Annual ACM Symposium on Theory of Computing (STOC)*, pages 163–172, 2015.
- [Che16] Jie Chen. How accurately should I compute implicit matrix-vector products when applying the Hutchinson trace estimator? *SIAM Journal on Scientific Computing*, 38(6):A3515–A3539, 2016.
- [CK20] Alice Cortinovis and Daniel Kressner. On randomized trace estimates for indefinite matrices with an application to determinants. *arXiv:2005.10009*, 2020.
- [CKSV18] David Cohen-Steiner, Weihao Kong, Christian Sohler, and Gregory Valiant. Approximating the spectrum of a graph. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1263–1271, 2018.
- [CMW15] Tony Cai, Zongming Ma, and Yihong Wu. Optimal estimation and rank detection for sparse spiked covariance matrices. *Probability theory and related fields*, 161:781–815, 2015.
- [CR12] Amit Chakrabarti and Oded Regev. An optimal lower bound on the communication complexity of gap-hamming-distance. *SIAM Journal on Computing*, 41(5):1299–1317, 2012.
- [CW09] Kenneth L. Clarkson and David P. Woodruff. Numerical linear algebra in the streaming model. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*, pages 205–214, 2009.
- [DEN⁺17] Kun Dong, David Eriksson, Hannes Nickisch, David Bindel, and Andrew Gordon Wilson. Scalable log determinants for Gaussian process kernel learning. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, pages 6327–6337, 2017.
- [DG03] Sanjoy Dasgupta and Anupam Gupta. An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22(1):60–65, 2003.
- [dlPGR07] José Antonio de la Peña, Ivan Gutman, and Juan Rada. Estimating the Estrada index. *Linear Algebra and its Applications*, 427(1):70–76, 2007.
- [DNPS16] Edoardo Di Napoli, Eric Polizzi, and Yousef Saad. Efficient estimation of eigenvalue counts in an interval. *Numerical Linear Algebra with Applications*, 2016.
- [EH08] Ernesto Estrada and Naomichi Hatano. Communicability in complex networks. *Phys. Rev. E*, 77:036111, Mar 2008.

- [EHB12] Ernesto Estrada, Naomichi Hatano, and Michele Benzi. The physics of communicability in complex networks. *Physics Reports*, 514(3):89 – 119, 2012.
- [EMM20] Tamás Erdélyi, Cameron Musco, and Christopher Musco. Fourier sparse leverage scores and approximate kernel learning. *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
- [Est00] Ernesto Estrada. Characterization of 3d molecular structure. *Chemical Physics Letters*, 319(5-6):713–718, 2000.
- [FKN90] Kaitai Fang, Samuel Kotz, and Kai Wang Ng. *Symmetric Multivariate and Related Distributions*. London: Chapman and Hall, 1990.
- [Gir87] Didier Girard. Un algorithme simple et rapide pour la validation croisee géénéralisée sur des problèmes de grande taille. Technical report, 1987.
- [GSO17] Arjun Singh Gambhir, Andreas Stathopoulos, and Kostas Orginos. Deflation as a method of variance reduction for estimating the trace of a matrix inverse. *SIAM Journal on Scientific Computing*, 39(2):A532–A558, 2017.
- [GSTV07] Anna C. Gilbert, Martin J. Strauss, Joel A. Tropp, and Roman Vershynin. One sketch for all: fast algorithms for compressed sensing. In David S. Johnson and Uriel Feige, editors, *Proceedings of the 39th Annual ACM Symposium on Theory of Computing, San Diego, California, USA, June 11-13, 2007*, pages 237–246. ACM, 2007.
- [Hig08] Nicholas J. Higham. *Functions of Matrices: Theory and Computation*. Society for Industrial and Applied Mathematics, 2008.
- [HMAS17] Insu Han, Dmitry Malioutov, Haim Avron, and Jinwoo Shin. Approximating the spectral sums of large-scale matrices using stochastic Chebyshev approximations. *SIAM Journal on Scientific Computing*, 2017.
- [HMS15] Insu Han, Dmitry Malioutov, and Jinwoo Shin. Large-scale log-determinant computation through stochastic Chebyshev expansions. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 908–917, 2015.
- [HMT11] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Rev.*, 53(2):217–288, 2011.
- [Hut90] Michael F. Hutchinson. A stochastic estimator of the trace of the influence matrix for Laplacian smoothing splines. *Communications in Statistics-Simulation and Computation*, 19(2):433–450, 1990.
- [IW03] Piotr Indyk and David Woodruff. Tight lower bounds for the distinct elements problem. In *Proceedings of the 44th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 2003.
- [JBYJHZ10] WU Jun, Mauricio Barahona, Tan Yue-Jin, and Deng Hong-Zhong. Natural connectivity of complex networks. *Chinese Physics Letters*, 27(7):078902, 2010.
- [Lin17] Lin Lin. Randomized estimation of spectral densities of large matrices made accurate. *Numerische Mathematik*, 136(1):183–213, 2017.

- [LSY16] Lin Lin, Yousef Saad, and Chao Yang. Approximating spectral densities of large matrices. *SIAM Review*, 58(1):34–65, 2016.
- [LZ21] Hanyu Li and Yuanyang Zhu. Randomized block krylov subspace methods for trace and log-determinant estimators. *BIT Numerical Mathematics*, 2021.
- [MM20] Cameron Musco and Christopher Musco. Projection-cost-preserving sketches: Proof strategies and constructions. [arXiv:2004.08434](https://arxiv.org/abs/2004.08434), 2020.
- [MMS18] Cameron Musco, Christopher Musco, and Aaron Sidford. Stability of the Lanczos method for matrix function approximation. In *Proceedings of the 29th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1605–1624, 2018.
- [MNS⁺18] Cameron Musco, Praneeth Netrapalli, Aaron Sidford, Shashanka Ubaru, and David P. Woodruff. Spectrum approximation beyond fast matrix multiplication: Algorithms and hardness. *Proceedings of the 9th Conference on Innovations in Theoretical Computer Science (ITCS)*, 2018.
- [MS77] Florence Jessie MacWilliams and Neil James Alexander Sloane. *The theory of error correcting codes*, volume 16. Elsevier, 1977.
- [MT20] Per-Gunnar Martinsson and Joel A. Tropp. Randomized numerical linear algebra: Foundations and algorithms. *Acta Numerica*, 29:403–572, 2020.
- [NM13] Markus Neteler and Helena Mitasova. *Open source GIS: a GRASS GIS approach*, volume 689. Springer Science & Business Media, 2013.
- [PT12] Rasmus Pagh and Charalampos E Tsourakakis. Colorful triangle counting and a mapreduce implementation. *Information Processing Letters*, 112(7):277–281, 2012.
- [PWBM18] Amelia Perry, Alexander Wein, Afonso Bandeira, and Ankur Moitra. Optimality and sub-optimality of PCA I: Spiked random matrix models. *Annals of Statistics*, 46:2416–2451, 10 2018.
- [RA15] Farbod Roosta-Khorasani and Uri M. Ascher. Improved bounds on sample size for implicit matrix trace estimators. *Foundations of Computational Mathematics*, 15(5):1187–1212, 2015.
- [Ras04] Carl Edward Rasmussen. Gaussian Processes in Machine Learning. In *Advanced Lectures on Machine Learning*, pages 63–71. Springer, 2004.
- [RV⁺13] Mark Rudelson, Roman Vershynin, et al. Hanson-Wright inequality and sub-Gaussian concentration. *Electronic Communications in Probability*, 18, 2013.
- [SAI17] Arvind K. Saibaba, Alen Alexanderian, and Ilse C. F. Ipsen. Randomized matrix-free trace and log-determinant estimators. *Numerische Mathematik*, 137(2):353–395, 2017.
- [Sar06] Tamas Sarlos. Improved approximation algorithms for large matrices via random projections. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 143–152, 2006.

- [SEAR18] Max Simchowitz, Ahmed El Alaoui, and Benjamin Recht. Tight query complexity lower bounds for PCA via finite sample deformed Wigner law. In *Proceedings of the 50th Annual ACM Symposium on Theory of Computing (STOC)*, pages 1249–1259, 2018.
- [She12] Alexander A. Sherstov. The communication complexity of gap hamming distance. *Theory of Computing*, 8(8):197–208, 2012.
- [SLO13] Andreas Stathopoulos, Jesse Laeuchli, and Kostas Orginos. Hierarchical probing for estimating the trace of the matrix inverse on toroidal lattices. *SIAM Journal on Scientific Computing*, 35(5):S299–S322, 2013.
- [SW05] Thomas Schank and Dorothea Wagner. Finding, counting and listing all triangles in large graphs, an experimental study. In *International Workshop on Experimental and Efficient Algorithms*, pages 606–609. Springer, 2005.
- [SWYZ19] Xiaoming Sun, David P. Woodruff, Guang Yang, and Jialin Zhang. Querying a matrix through matrix-vector products. In *Proceedings of the 46th International Colloquium on Automata, Languages and Programming (ICALP)*, volume 132, pages 94:1–94:16, 2019.
- [TS11] Jok M. Tang and Yousef Saad. Domain-decomposition-type methods for computing the diagonal of a matrix inverse. *SIAM Journal on Scientific Computing*, 33(5):2823–2847, 2011.
- [Tso08] Charalampos E Tsourakakis. Fast counting of triangles in large real networks without counting: Algorithms and laws. In *2008 Eighth IEEE International Conference on Data Mining*, pages 608–617, 2008.
- [UCS17] Shashanka Ubaru, Jie Chen, and Yousef Saad. Fast estimation of $\text{tr}(f(A))$ via stochastic Lanczos quadrature. *SIAM Journal on Matrix Analysis and Applications*, 38(4):1075–1099, 2017.
- [US18] Shashanka Ubaru and Yousef Saad. Applications of trace estimation techniques. In *High Performance Computing in Science and Engineering*, pages 19–33, 2018.
- [Vid12] Thomas Vidick. A concentration inequality for the overlap of a vector on a large set, with application to the communication complexity of the gap-hamming-distance problem. *Chicago Journal of Theoretical Computer Science*, 2012.
- [Woo14] David P. Woodruff. Sketching as a tool for numerical linear algebra. *Foundations and Trends in Theoretical Computer Science*, 10(1–2):1–157, 2014.
- [WR96] Christopher K. I. Williams and Carl Edward Rasmussen. Gaussian Processes for Regression. In *Advances in Neural Information Processing Systems 9 (NeurIPS)*, pages 514–520, 1996.
- [WSMB20] Sheng Wang, Yuan Sun, Christopher Musco, and Zhifeng Bao. Route planning for robust transit networks: When connectivity matters. *Preprint*, 2020.
- [WWZ14] Karl Wimmer, Yi Wu, and Peng Zhang. Optimal query complexity for estimating the trace of a matrix. In *Proceedings of the 41st International Colloquium on Automata, Languages and Programming (ICALP)*, pages 1051–1062, 2014.

A Proof of Lemma 2

We start by stating the Hanson-Wright inequality for i.i.d sub-Gaussian random variables:

Imported Theorem 15 ([RV⁺13]). *Let $\mathbf{x} \in \mathbb{R}^n$ be a vector of mean 0, i.i.d. sub-Gaussian random variables with constant sub-Gaussian parameter C . Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a matrix. Then, there exists a constant c only depending on C such that for every $t \geq 0$,*

$$\Pr \{ |\mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbb{E}[\mathbf{x}^T \mathbf{A} \mathbf{x}]| > t \} \leq 2 \exp \left(-c \cdot \min \left\{ \frac{t^2}{\|\mathbf{A}\|_F^2}, \frac{t}{\|\mathbf{A}\|_2} \right\} \right).$$

Above, $\|\mathbf{A}\|_2 = \max_x \|\mathbf{A} \mathbf{x}\|_2 / \|\mathbf{x}\|_2$ denotes the spectral norm. We refer the reader to [RV⁺13] for a formal definition of sub-Gaussian random variables: both normal $\mathcal{N}(0, 1)$ random variables and ± 1 random variables are sub-Gaussian with constant C .

Lemma 2 Restated. *Let $\mathbf{A} \in \mathbb{R}^{d \times d}$, $\delta \in (0, 1/2]$, $\ell \in \mathbb{N}$. Let $H_\ell(\mathbf{A})$ be the ℓ -query Hutchinson estimator defined in (1), implemented with mean 0, i.i.d. sub-Gaussian random variables with constant sub-Gaussian parameter. For fixed constants c, C , if $\ell > c \log(1/\delta)$, then with prob. $1 - \delta$,*

$$|H_\ell(\mathbf{A}) - \text{tr}(\mathbf{A})| \leq C \sqrt{\frac{\log(1/\delta)}{\ell}} \|\mathbf{A}\|_F.$$

Proof. Let $\bar{\mathbf{A}} \in \mathbb{R}^{\ell d \times \ell d}$ be a block-diagonal matrix formed from ℓ repetitions of $\mathbf{A}$:

$$\bar{\mathbf{A}} := \begin{bmatrix} \mathbf{A} & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{A} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{A} \end{bmatrix}.$$

Let $\mathbf{G} \in \mathbb{R}^{d \times \ell}$ be as in (1). Let $\mathbf{g}_i$ be $\mathbf{G}$'s i^{th} column and let $\mathbf{g} = [\mathbf{g}_1, \dots, \mathbf{g}_\ell] \in \mathbb{R}^{d\ell}$ be a vectorization of $\mathbf{G}$. We have that $\ell \cdot H_\ell(\mathbf{A}) = \text{tr}(\mathbf{G}^T \bar{\mathbf{A}} \mathbf{G}) = \mathbf{g}^T \bar{\mathbf{A}} \mathbf{g}$. So, by **Imported Theorem 15**,

$$\Pr \{ |\mathbf{g}^T \bar{\mathbf{A}} \mathbf{g} - \mathbb{E}[\mathbf{g}^T \bar{\mathbf{A}} \mathbf{g}]| > t \} \leq 2 \exp \left(-c \cdot \min \left\{ \frac{t^2}{\|\bar{\mathbf{A}}\|_F^2}, \frac{t}{\|\bar{\mathbf{A}}\|_2} \right\} \right). \quad (3)$$

We let $t' = t/\ell$, and substitute $\mathbb{E}[\mathbf{g}^T \bar{\mathbf{A}} \mathbf{g}] = \text{tr}(\bar{\mathbf{A}}) = \ell \text{tr}(\mathbf{A})$, $\|\bar{\mathbf{A}}\|_F^2 = \ell \|\mathbf{A}\|_F^2$, and $\|\bar{\mathbf{A}}\|_2 = \|\mathbf{A}\|_2$ into (3) to get:

$$\Pr \{ |H_\ell(\mathbf{A}) - \text{tr}(\mathbf{A})| > t' \} \leq 2 \exp \left(-c \min \left\{ \frac{\ell t'^2}{\|\mathbf{A}\|_F^2}, \frac{\ell t'}{\|\mathbf{A}\|_2} \right\} \right).$$

Now, taking $t' = \sqrt{\frac{\ln(2/\delta)}{c\ell}} \|\mathbf{A}\|_F$, we have:

$$\Pr \left\{ |H_\ell(\mathbf{A}) - \text{tr}(\mathbf{A})| > \sqrt{\frac{\ln(2/\delta)}{c\ell}} \|\mathbf{A}\|_F \right\} \leq 2 \exp \left(-\min \left\{ \log(2/\delta), \sqrt{c\ell \log(2/\delta)} \frac{\|\mathbf{A}\|_F}{\|\mathbf{A}\|_2} \right\} \right)$$

Since $\frac{\|\mathbf{A}\|_F}{\|\mathbf{A}\|_2} \geq 1$, if we take $\ell \geq \ln(2/\delta)/c$, we have that the minimum takes value $\log(2/\delta)$, so

$$\Pr \left\{ |H_\ell(\mathbf{A}) - \text{tr}(\mathbf{A})| > \sqrt{\frac{\ln(2/\delta)}{c\ell}} \|\mathbf{A}\|_F \right\} \leq \delta.$$

The final result follows from noting that $\ln(2/\delta) \leq 2 \ln(1/\delta)$ for $\delta \leq 1/2$. □

B Proof of Theorem 8

To prove our non-adaptive lower bound for the real RAM model we first introduce a simple testing problem which we reduce to estimating the trace of a PSD matrix $\mathbf{A}$ to $(1 \pm \varepsilon)$ relative error:

Problem 2. Fix $d, n \in \mathbb{N}$ such that $d \geq n$ and $n := \frac{1}{\varepsilon}$ for $\varepsilon \in (0, 1]$. Let $\mathbf{D}_1 = \mathbf{I}_n$ and $\mathbf{D}_2 = \begin{pmatrix} \mathbf{I}_{n-1} & \\ & 0 \end{pmatrix}$.⁵ Consider $\mathbf{A} = \mathbf{G}^T \mathbf{D} \mathbf{G}$ generated by selecting $\mathbf{G} \in \mathbb{R}^{n \times d}$ with i.i.d. random Gaussian $\mathcal{N}(0, 1)$ entries and $\mathbf{D} = \mathbf{D}_1$ or $\mathbf{D} = \mathbf{D}_2$ with equal probability. Then consider any algorithm which fixes a query matrix $\mathbf{U} \in \mathbb{R}^{d \times m}$, observes $\mathbf{A} \mathbf{U} \in \mathbb{R}^{d \times m}$, and guesses if $\mathbf{D} = \mathbf{D}_1$ or $\mathbf{D} = \mathbf{D}_2$.

The reduction from Problem 2 to relative error trace estimation is as follows:

Lemma 16. For any $\varepsilon \in (0, 1]$ and sufficient large d , if a randomized algorithm $\mathcal{A}$ can estimate the trace of any $d \times d$ PSD matrix to relative error $1 \pm \frac{\varepsilon}{4}$ with success probability $\geq \frac{3}{4}$ using m queries, then $\mathcal{A}$ can be used to solve Problem 2 with success probability $\geq \frac{2}{3}$ using m queries.

Proof. To solve Problem 2 we simply apply $\mathcal{A}$ to the matrix $\mathbf{A} = \mathbf{G}^T \mathbf{D} \mathbf{G}$ and guess $\mathbf{D}_1$ if the trace is closer to $\frac{d}{\varepsilon}$ and $\mathbf{D}_2$ if it's closer to $\frac{d}{\varepsilon} - d$. To see that this succeeds with probability $2/3$, we first need to understand the trace of $\mathbf{A}$. To do so, note that $\text{tr}(\mathbf{A}) = \text{tr}(\mathbf{G}^T \mathbf{D} \mathbf{G})$ is simply a scaled Hutchinson estimate for $\text{tr}(\mathbf{D})$, i.e. $\text{tr}(\mathbf{G}^T \mathbf{D} \mathbf{G}) = d \cdot \text{H}_d(\mathbf{D})$. So, via Lemma 2, for large enough d we have that with probability $\geq \frac{11}{12}$ both of the following hold:

$$\frac{1}{d} \text{tr}(\mathbf{G}^T \mathbf{D}_1 \mathbf{G}) \geq \left(1 - \frac{\varepsilon}{4}\right) \text{tr}(\mathbf{D}_1) \quad \text{and} \quad \frac{1}{d} \text{tr}(\mathbf{G}^T \mathbf{D}_2 \mathbf{G}) \leq \left(1 + \frac{\varepsilon}{4}\right) \text{tr}(\mathbf{D}_2).$$

Additionally, with probability $\frac{3}{4}$, $\mathcal{A}$ computes an approximation Z with $(1 - \frac{\varepsilon}{4}) \text{tr}(\mathbf{A}) \leq Z \leq (1 + \frac{\varepsilon}{4}) \text{tr}(\mathbf{A})$. By a union bound, all of the above events happen with probability $\geq \frac{2}{3}$. If $\mathbf{D} = \mathbf{D}_1$:

$$Z \geq (1 - \frac{\varepsilon}{4}) \text{tr}(\mathbf{A}) \geq (1 - \frac{\varepsilon}{4})^2 \cdot d \cdot \text{tr}(\mathbf{D}_1) > (1 - \frac{\varepsilon}{2}) \cdot \frac{d}{\varepsilon}.$$

On the other hand, if $\mathbf{D} = \mathbf{D}_2$,

$$\begin{aligned} Z &\leq (1 + \frac{\varepsilon}{4}) \text{tr}(\mathbf{A}) \leq (1 + \frac{\varepsilon}{4})^2 \cdot d \cdot \text{tr}(\mathbf{D}_2) \\ &= (1 + \frac{\varepsilon}{4})^2 \cdot (1 - \varepsilon) \cdot \frac{d}{\varepsilon} < (1 - \frac{\varepsilon}{2}) \cdot \frac{d}{\varepsilon}. \end{aligned}$$

Thus, with probability $2/3$, Z is closer to $\frac{d}{\varepsilon}$ when $\mathbf{D} = \mathbf{D}_1$ and closer to $\frac{d}{\varepsilon} - d$ when $\mathbf{D} = \mathbf{D}_2$, so the proposed scheme guesses correctly. $\square$

In the remainder of the section we show that Problem 2 requires $\Omega(1/\varepsilon)$ queries, which combined with Lemma 16 proves our main lower bound, Theorem 8. Throughout, we let $\mathbf{X} \stackrel{\text{dist}}{=} \mathbf{Y}$ denote that $\mathbf{X}$ and $\mathbf{Y}$ are identically distributed. We first argue that for Problem 2, the non-adaptive query matrix $\mathbf{U}$ might as well be chosen to be the first m standard basis vectors.

Lemma 17. For Problem 2, without loss of generality, we may assume that the query matrix $\mathbf{U}$ equals $\mathbf{U} = \mathbf{E}_m = [\mathbf{e}_1 \ \dots \ \mathbf{e}_m]$, the first m standard basis vectors.

Proof. First, we may assume without loss of generality that $\mathbf{U}$ is orthonormal, since if it were not, we could simply reconstruct the queries $\mathbf{A} \mathbf{U}$ by querying $\mathbf{A}$ with an orthonormal basis for the columns of $\mathbf{U}$. Next, by rotational invariance of the Gaussian distribution, if $\mathbf{G} \in \mathbb{R}^{n \times d}$ is an i.i.d.

⁵Here $\mathbf{I}_r$ denotes an $r \times r$ identity matrix.

$\mathcal{N}(0, 1)$ matrix, and $\mathbf{Q} \in \mathbb{R}^{d \times d}$ is any orthogonal matrix, then $\mathbf{G}\mathbf{Q}$ is distributed identically to $\mathbf{G}$. Let $\bar{\mathbf{U}} \in \mathbb{R}^{d \times d-m}$ be any orthonormal span for the nullspace of $\mathbf{U}$, so that $\mathbf{Q} := [\mathbf{U} \ \bar{\mathbf{U}}]$ is orthogonal. We have that $\mathbf{Q}\mathbf{G}^T \mathbf{D}\mathbf{G}\mathbf{E}_m \stackrel{\text{dist}}{=} \mathbf{Q}\mathbf{Q}^T \mathbf{G}^T \mathbf{D}\mathbf{G}\mathbf{Q}\mathbf{E}_m = \mathbf{G}^T \mathbf{D}\mathbf{G}\mathbf{U}$. So, using the result $\mathbf{G}^T \mathbf{D}\mathbf{G}\mathbf{E}_m$ of querying with matrix $\mathbf{E}_m$, we can just multiply by $\mathbf{Q}$ on the left to obtain a set of vectors that has the same distribution as if $\mathbf{U}$ had been used as a query matrix. $\square$

With Lemma 17 in place, we are able to reduce Problem 2 to a simpler testing problem on distinguishing m random vectors drawn from normal distributions with different covariance matrices:

Problem 3. Let $n = \frac{1}{\varepsilon}$ and let $\mathbf{z} \in \mathbb{R}^n$ be a uniformly random unit vector. Let $\mathbf{N} \in \mathbb{R}^{n \times m}$ contain m i.i.d. random Gaussian vectors drawn from an n -dimensional Gaussian distribution, $\mathcal{N}(\mathbf{0}, \mathbf{C})$, where the covariance matrix $\mathbf{C}$ either equals $\mathbf{I}$ or $\mathbf{I} - \mathbf{z}\mathbf{z}^T$, with equality probability. The goal is to use $\mathbf{N}$ to distinguish, with probability $> \frac{1}{2}$ what the true identity of $\mathbf{C}$ is.

Lemma 18. Let $\mathcal{A}$ be an algorithm that solves Problem 2 with m queries and success probability p . Then $\mathcal{A}$ can be used to solve Problem 3 with m Gaussian samples and the same success probability.

Proof. By Lemma 17, it suffices to show how to use the observed matrix $\mathbf{N}$ in Problem 3 to create a sample from the distribution $\mathbf{G}^T \mathbf{D}\mathbf{G}\mathbf{E}_m$ where $\mathbf{G} \in \mathbb{R}^{n \times d}$ has i.i.d. $\mathcal{N}(0, 1)$ entries. Specifically, we claim that, if we sample $\mathbf{L} \in \mathbb{R}^{n \times (d-m)}$ with i.i.d. $\mathcal{N}(0, 1)$ entries, and compute

$$\mathbf{M} = \begin{bmatrix} \mathbf{N}^T \mathbf{N} \\ \mathbf{L}^T \mathbf{N} \end{bmatrix},$$

then $\mathbf{M}$ is identically distributed to $\mathbf{G}^T \mathbf{D}\mathbf{G}\mathbf{E}_m$. I.e, if we let $\mathbf{G}_m \in \mathbb{R}^{n \times m}$ contain the first m columns of $\mathbf{G}$ and let $\mathbf{G}_{d-m}$ contain the remaining $d - m$ columns, our goal is to show that $\mathbf{M} \stackrel{\text{dist}}{=} [\mathbf{G}_m \ \mathbf{G}_{d-m}]^T \mathbf{D}\mathbf{G}_m$.

To see this is the case, let $\mathbf{Z} \in \mathbb{R}^{n \times n}$ be a uniformly random orthogonal matrix and let $\mathbf{D}_1, \mathbf{D}_2$ be as in Problem 2. The first observation is that $\mathbf{N} \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$ is identically distributed to $\mathbf{Z}\mathbf{D}\mathbf{S}$ where $\mathbf{S} \in \mathbb{R}^{d \times m}$ has standard normal entries $\sim \mathcal{N}(0, 1)$ and $\mathbf{D} = \mathbf{D}_1$ or $\mathbf{D} = \mathbf{D}_2$ with equal probability. This follows simply from the fact that $\mathbf{Z}\mathbf{D}_1\mathbf{D}_1\mathbf{Z}^T = \mathbf{I}$ and $\mathbf{Z}\mathbf{D}_2\mathbf{D}_2\mathbf{Z}^T = \mathbf{I} - \mathbf{z}_n\mathbf{z}_n^T$, where $\mathbf{z}_n$ is the last row of $\mathbf{Z}$, which is a uniformly random unit vector. It follows that $\mathbf{N}^T \mathbf{N} \stackrel{\text{dist}}{=} \mathbf{S}^T \mathbf{D}^T \mathbf{Z}^T \mathbf{Z} \mathbf{D} \mathbf{S} = \mathbf{S}^T \mathbf{D} \mathbf{S} \stackrel{\text{dist}}{=} \mathbf{G}_m^T \mathbf{D}\mathbf{G}_m$. Next, observe that $\mathbf{L}^T \mathbf{Z}$ is independent of $\mathbf{G}$ and has i.i.d. $\mathcal{N}(0, 1)$ entries since $\mathbf{Z}$ is orthogonal (and Gaussians are rotationally invariant). So, $\mathbf{L}^T \mathbf{N} \stackrel{\text{dist}}{=} \mathbf{L}^T \mathbf{Z} \mathbf{D} \mathbf{S} \stackrel{\text{dist}}{=} \mathbf{G}_{d-m}^T \mathbf{D}\mathbf{G}$ and overall:

$$\mathbf{M} = \begin{bmatrix} \mathbf{N}^T \mathbf{N} \\ \mathbf{L}^T \mathbf{N} \end{bmatrix} \stackrel{\text{dist}}{=} \begin{bmatrix} \mathbf{G}_m^T \mathbf{D}\mathbf{G}_m \\ \mathbf{G}_{d-m}^T \mathbf{D}\mathbf{G}_m \end{bmatrix} = [\mathbf{G}_m \ \mathbf{G}_{d-m}]^T \mathbf{D}\mathbf{G}_m. \quad \square$$

Finally, we directly prove a lower bound on the number of samples m required to solve Problem 3, and thus, via Lemma 18, Problem 2. Combined with Lemma 16, this immediately yields our main lower bound on non-adaptive trace estimation, Theorem 8.

Lemma 19. If $m < \frac{c}{\varepsilon}$ for a fixed constant c , then Problem 3 cannot be solved with probability $\geq \frac{2}{3}$.

Proof. The proof follows from existing work on lower bounds for learning “negatively spiked” covariance matrices [CMW15, PWBM18]. Let $\mathcal{P}$ be the distribution of $\mathbf{N}$ in Problem 3, conditioned on $\mathbf{C} = \mathbf{I}$, and let $\mathcal{Q}$ be the distribution conditioned on $\mathbf{C} = \mathbf{I} - \mathbf{z}\mathbf{z}^T$. These distributions fall into the spiked covariance model of [PWBM18], specifically the negatively spiked Wishart model (see

Defn. 5.1 in [PWBM18]) with spike size $\beta = -1$, and spike distribution $\mathcal{X}$ the uniform distribution over unit vectors in $\mathbb{R}^n$. Let $D_{\chi^2}(\mathcal{P} \parallel \mathcal{Q})$ denote the χ^2 divergence between $\mathcal{P}$ and $\mathcal{Q}$. Specifically,

$$D_{\chi^2}(\mathcal{Q} \parallel \mathcal{P}) = \int_{\mathbf{X} \in \mathbb{R}_{d \times m}} \left(\frac{\mathcal{Q}(\mathbf{X})}{\mathcal{P}(\mathbf{X})} \right)^2 \mathcal{P}(\mathbf{X}) d\mathbf{X} - 1.$$

We have $D_{KL}(\mathcal{Q} \parallel \mathcal{P}) \leq D_{\chi^2}(\mathcal{Q} \parallel \mathcal{P})$, so to prove that $\mathcal{P}, \mathcal{Q}$ cannot be distinguished with good probability, it suffices to prove an upper bound on $D_{\chi^2}(\mathcal{Q} \parallel \mathcal{P})$. In [CMW15] (Lemma 7) it is proven that, letting $\mathbf{v}$ and $\mathbf{v}'$ be independent random unit vectors in $\mathbb{R}^n$,

$$D_{\chi^2}(\mathcal{Q} \parallel \mathcal{P}) = \mathbb{E}_{\mathbf{v}, \mathbf{v}'} \left[(1 - \langle \mathbf{v}, \mathbf{v}' \rangle)^{-m/2} \right] - 1. \quad (4)$$

Equation (4) uses the notation of Prop. 5.11 in [PWBM18], which restates and proves a slightly less general form of the equality from [CMW15]. Our goal is to prove that the expectation term in (4) is $\leq 1 + C$ for some small constant C when $m = \frac{c}{\varepsilon} = cn$ for a sufficiently small constant c .

We first note that $\langle \mathbf{v}, \mathbf{v}' \rangle$ is identically distributed to $x \in [-1, 1]$ where x is the first entry in a random unit vector in $\mathbb{R}^n$. It is well known that $\frac{x+1}{2}$ is distributed according to a beta distribution with parameters $\alpha = \beta = \frac{n-1}{2}$ [FKN90]. Specifically, this gives that x has density:

$$p(x) = \frac{\Gamma(2\alpha)}{2\Gamma(\alpha)^2} \cdot \left(\frac{1-x^2}{4} \right)^{\alpha-1}.$$

Plugging this density back in to the expectation term in (4) we obtain:

$$\begin{aligned} \mathbb{E}_{\mathbf{v}, \mathbf{v}'} \left[(1 - \langle \mathbf{v}, \mathbf{v}' \rangle)^{-m/2} \right] &= \int_{-1}^1 \frac{\Gamma(2\alpha)}{2\Gamma(\alpha)^2} \cdot \left(\frac{1-x^2}{4} \right)^{\alpha-1} (1-x^2)^{-m/2} dx \\ &= \int_{-1}^1 \frac{\Gamma(2\alpha)}{2\Gamma(\alpha)^2} \cdot \left(\frac{1}{4} \right)^{\alpha-1} (1-x^2)^{\alpha-1-m/2} dx \end{aligned}$$

Assume without loss of generality that n is an odd integer, and thus $\alpha = \frac{n-1}{2}$ is an integer. Let $m/2 = c\alpha$ for some constant $c \ll 1$ such that $c\alpha$ is an integer and thus $(1-c)\alpha$ is an integer. Then:

$$\begin{aligned} \mathbb{E}_{\mathbf{v}, \mathbf{v}'} \left[(1 - \langle \mathbf{v}, \mathbf{v}' \rangle)^{-m/2} \right] &= \frac{\Gamma(2\alpha)}{\Gamma(\alpha)^2} \cdot \left(\frac{1}{4} \right)^{c\alpha} \cdot \int_{-1}^1 \frac{\Gamma(2(1-c)\alpha)}{2\Gamma((1-c)\alpha)^2} \left(\frac{1}{4} \right)^{(1-c)\alpha-1} (1-x^2)^{(1-c)\alpha-1} dx \\ &= \frac{\Gamma(2\alpha)}{\Gamma(\alpha)^2} \cdot \left(\frac{1}{4} \right)^{c\alpha}, \end{aligned} \quad (5)$$

where the equality follows because the term being integrated is the density of x where $\frac{x+1}{2}$ is distributed according to a beta distribution with parameters $(1-c)\alpha, (1-c)\alpha$. Since we have chosen parameters such that α is a positive integer, we have:

$$\frac{\Gamma(2\alpha)}{\Gamma(\alpha)^2} = \frac{(2\alpha-1)!}{(\alpha-1)!(\alpha-1)!} = \frac{\alpha}{2} \cdot \binom{2\alpha}{\alpha}.$$

Similarly, $\frac{\Gamma(2(1-c)\alpha)}{\Gamma((1-c)\alpha)^2} = \frac{(1-c)\alpha}{2} \cdot \binom{2(1-c)\alpha}{(1-c)\alpha}$. Each of the binomial coefficients in these expressions is a central binomial coefficient (i.e., proportional to a Catalan number), and we can use well known

methods like Stirling's approximation to bound them. In particular, we employ a bound given in Lemma 7 of [MS77], which gives $\frac{1}{2} \frac{4^z}{\sqrt{z}} \leq \binom{2z}{z} \leq \frac{1}{\sqrt{\pi}} \frac{4^z}{\sqrt{z}}$. for any integer z . Accordingly, we have

$$\begin{aligned} \frac{\frac{\Gamma(2\alpha)}{\Gamma(\alpha)^2}}{\frac{\Gamma(2(1-c)\alpha)}{\Gamma((1-c)\alpha)^2}} &= \frac{1}{1-c} \cdot \frac{\binom{2\alpha}{\alpha}}{\binom{2(1-c)\alpha}{(1-c)\alpha}} \\ &\leq \frac{1}{1-c} \cdot \frac{1/\sqrt{\pi}}{1/2} \frac{4^\alpha}{4^{(1-c)\alpha}} \cdot \sqrt{\frac{(1-c)\alpha}{\alpha}} \\ &= \frac{2}{\sqrt{\pi(1-c)}} \cdot 4^{c\alpha}. \end{aligned}$$

Plugging into (5) and requiring $c \leq .1$ we have:

$$\mathbb{E}_{\mathbf{v}, \mathbf{v}'} \left[(1 - \langle \mathbf{v}, \mathbf{v}' \rangle^2)^{-m/2} \right] \leq \frac{2}{\sqrt{\pi} \cdot .9} < \frac{6}{5}.$$

It follows that $D_{KL}(\mathcal{Q} \parallel \mathcal{P}) \leq D_{\chi^2}(\mathcal{Q} \parallel \mathcal{P}) \leq \frac{6}{5} - 1 = \frac{1}{5}$, and thus by Pinsker's inequality that

$$D_{TV}(\mathcal{Q}, \mathcal{P}) \leq \frac{1}{\sqrt{10}} < \frac{1}{3}.$$

Thus, no algorithm can solve **Problem 3** with probability $\geq \frac{1}{2} + \frac{1/3}{2} = \frac{2}{3}$, completing the lemma. $\square$